

第 2 章 ROS通信机制

机器人是一种高度复杂的系统性实现，在机器人上可能集成各种传感器(雷达、摄像头、GPS...)以及运动控制实现，为了解耦合，在ROS中每一个功能点都是一个单独的进程，每一个进程都是独立运行的。更确切的讲，**ROS**是进程（也称为*Nodes*）的分布式框架。因为这些进程甚至还可分布于不同主机，不同主机协同工作，从而分散计算压力。不过随之也有一个问题: 不同的进程是如何通信的？也即不同进程间如何实现数据交换的？在此我们就需要介绍一下ROS中的通信机制了。

ROS 中的基本通信机制主要有如下三种实现策略:

- 话题通信(发布订阅模式)
- 服务通信(请求响应模式)
- 参数服务器(参数共享模式)

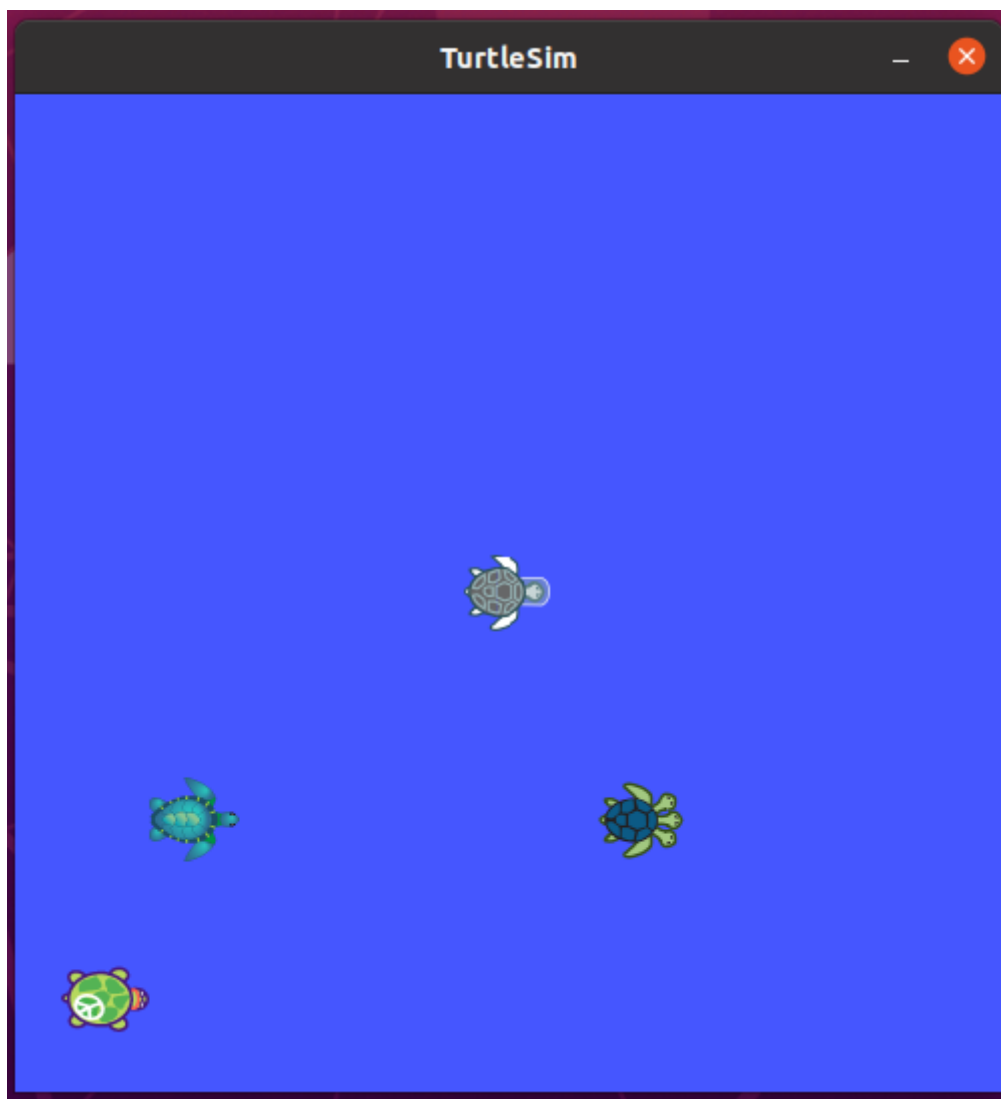
本章的主要内容就是介绍各个通信机制的应用场景、理论模型、代码实现以及相关操作命令。本章预期达成学习目标如下:

- 能够熟练介绍ROS中常用的通信机制
- 能够理解ROS中每种通信机制的理论模型
- 能够以代码的方式实现各种通信机制对应的案例
- 能够熟练使用ROS中的一些操作命令
- 能够独立完成相关实操案例

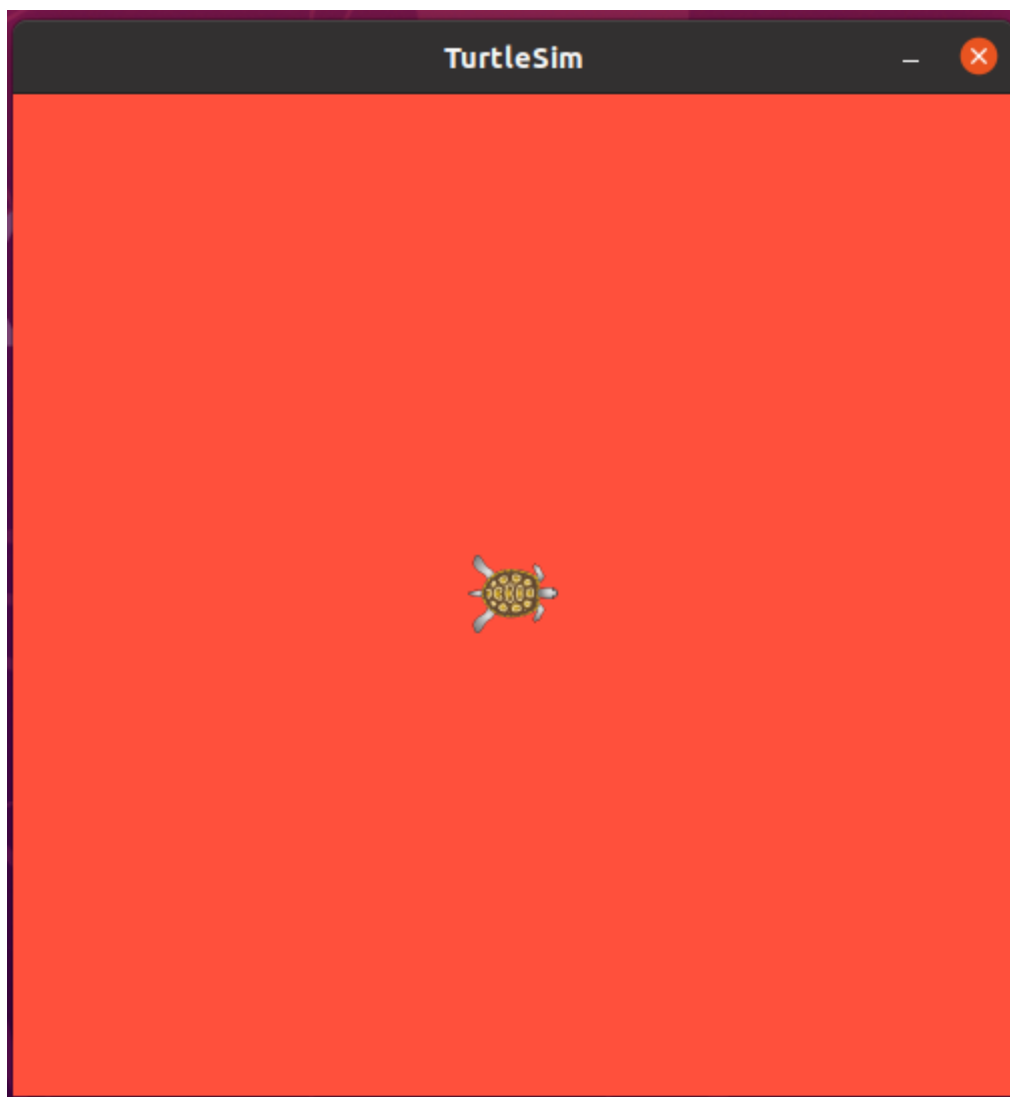
案例演示:

1.话题演示案例:

控制小乌龟做圆周运动



3.参数演示案例:改变乌龟窗口的背景颜色



2.1 话题通信

话题通信是ROS中使用频率最高的一种通信模式，话题通信是基于发布订阅模式的，也即：一个节点发布消息，另一个节点订阅该消息。话题通信的应用场景也极其广泛，比如下面一个常见场景：

机器人在执行导航功能，使用的传感器是激光雷达，机器人会采集激光雷达感知到的信息并计算，然后生成运动控制信息驱动机器人底盘运动。

在上述场景中，就不止一次使用到了话题通信。

- 以激光雷达信息的采集处理为例，在 ROS 中有一个节点需要时时的发布当前雷达采集到的数据，导航模块中也有节点会订阅并解析雷达数据。
- 再以运动消息的发布为例，导航模块会根据传感器采集的数据时时的计算出运动控制信息并发布给底盘，底盘也可以有一个节点订阅运动信息并最终转换成控制电机的脉冲信号。

以此类推，像雷达、摄像头、GPS..... 等等一些传感器数据的采集，也都是使用了话题通信，换言之，话题通信适用于不断更新的数据传输相关的应用场景。

概念

以发布订阅的方式实现不同节点之间数据交互的通信模式。

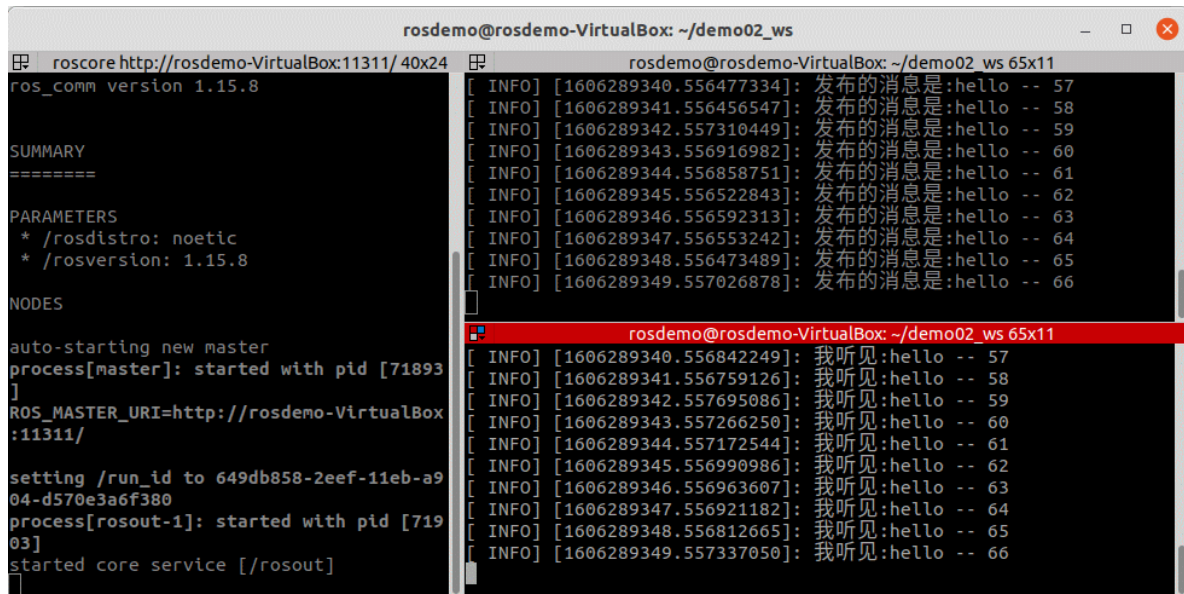
作用

用于不断更新的、少逻辑处理的数据传输场景。

案例

1.实现最基本的发布订阅模型，发布方以固定频率发送一段文本，订阅方接收文本并输出。

(2.1.2 -- 2.1.3)



```
rosdemo@rosdemo-VirtualBox: ~/demo02_ws
roscore http://rosdemo-VirtualBox:11311/ 40x24
ros_comm version 1.15.8

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.8

NODES

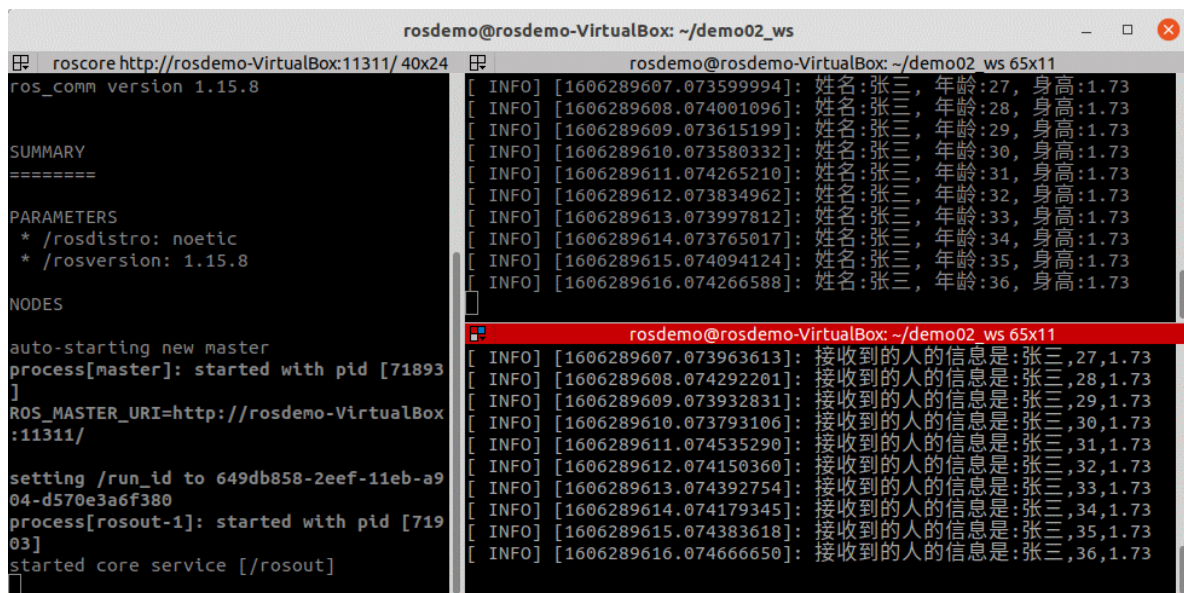
auto-starting new master
process[master]: started with pid [71893]
ROS_MASTER_URI=http://rosdemo-VirtualBox:11311/

setting /run_id to 649db858-2eef-11eb-a904-d570e3a6f380
process[rosout-1]: started with pid [71903]
started core service [/rosout]

rosdemo@rosdemo-VirtualBox: ~/demo02_ws 65x11
[ INFO] [1606289340.556477334]: 发布的消息是:hello -- 57
[ INFO] [1606289341.556456547]: 发布的消息是:hello -- 58
[ INFO] [1606289342.557310449]: 发布的消息是:hello -- 59
[ INFO] [1606289343.556916982]: 发布的消息是:hello -- 60
[ INFO] [1606289344.556858751]: 发布的消息是:hello -- 61
[ INFO] [1606289345.556522843]: 发布的消息是:hello -- 62
[ INFO] [1606289346.556592313]: 发布的消息是:hello -- 63
[ INFO] [1606289347.556553242]: 发布的消息是:hello -- 64
[ INFO] [1606289348.556473489]: 发布的消息是:hello -- 65
[ INFO] [1606289349.557026878]: 发布的消息是:hello -- 66

rosdemo@rosdemo-VirtualBox: ~/demo02_ws 65x11
[ INFO] [1606289340.556842249]: 我听见:hello -- 57
[ INFO] [1606289341.556759126]: 我听见:hello -- 58
[ INFO] [1606289342.557695086]: 我听见:hello -- 59
[ INFO] [1606289343.557266250]: 我听见:hello -- 60
[ INFO] [1606289344.557172544]: 我听见:hello -- 61
[ INFO] [1606289345.556990986]: 我听见:hello -- 62
[ INFO] [1606289346.556963607]: 我听见:hello -- 63
[ INFO] [1606289347.556921182]: 我听见:hello -- 64
[ INFO] [1606289348.556812665]: 我听见:hello -- 65
[ INFO] [1606289349.557337050]: 我听见:hello -- 66
```

2.实现对自定义消息的发布与订阅。(2.1.4 -- 2.1.6)



```
rosdemo@rosdemo-VirtualBox: ~/demo02_ws
roscore http://rosdemo-VirtualBox:11311/ 40x24
ros_comm version 1.15.8

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.8

NODES

auto-starting new master
process[master]: started with pid [71893]
ROS_MASTER_URI=http://rosdemo-VirtualBox:11311/

setting /run_id to 649db858-2eef-11eb-a904-d570e3a6f380
process[rosout-1]: started with pid [71903]
started core service [/rosout]

rosdemo@rosdemo-VirtualBox: ~/demo02_ws 65x11
[ INFO] [1606289607.073599994]: 姓名:张三, 年龄:27, 身高:1.73
[ INFO] [1606289608.074001096]: 姓名:张三, 年龄:28, 身高:1.73
[ INFO] [1606289609.073615199]: 姓名:张三, 年龄:29, 身高:1.73
[ INFO] [1606289610.073580332]: 姓名:张三, 年龄:30, 身高:1.73
[ INFO] [1606289611.074265210]: 姓名:张三, 年龄:31, 身高:1.73
[ INFO] [1606289612.073834962]: 姓名:张三, 年龄:32, 身高:1.73
[ INFO] [1606289613.073997812]: 姓名:张三, 年龄:33, 身高:1.73
[ INFO] [1606289614.073765017]: 姓名:张三, 年龄:34, 身高:1.73
[ INFO] [1606289615.074094124]: 姓名:张三, 年龄:35, 身高:1.73
[ INFO] [1606289616.074266588]: 姓名:张三, 年龄:36, 身高:1.73

rosdemo@rosdemo-VirtualBox: ~/demo02_ws 65x11
[ INFO] [1606289607.073963613]: 接收到的人的信息是:张三,27,1.73
[ INFO] [1606289608.074292201]: 接收到的人的信息是:张三,28,1.73
[ INFO] [1606289609.073932831]: 接收到的人的信息是:张三,29,1.73
[ INFO] [1606289610.073793106]: 接收到的人的信息是:张三,30,1.73
[ INFO] [1606289611.074535290]: 接收到的人的信息是:张三,31,1.73
[ INFO] [1606289612.074150360]: 接收到的人的信息是:张三,32,1.73
[ INFO] [1606289613.074392754]: 接收到的人的信息是:张三,33,1.73
[ INFO] [1606289614.074179345]: 接收到的人的信息是:张三,34,1.73
[ INFO] [1606289615.074383618]: 接收到的人的信息是:张三,35,1.73
[ INFO] [1606289616.074666650]: 接收到的人的信息是:张三,36,1.73
```

另请参考:

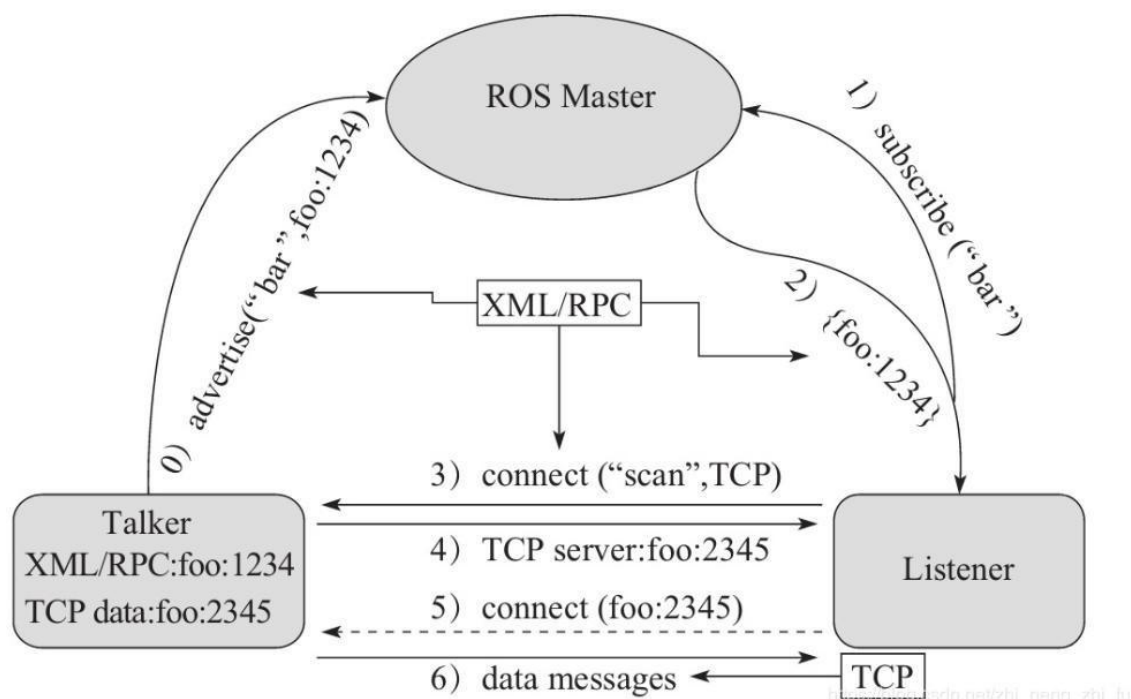
- <http://wiki.ros.org/ROS/Tutorials/CreatingMsgAndSrv>
- <http://wiki.ros.org/ROS/Tutorials/WritingPublisherSubscriber%28c%2B%2B%29>
- <http://wiki.ros.org/ROS/Tutorials/WritingPublisherSubscriber%28python%29>

2.1.1 理论模型

话题通信实现模型是比较复杂的，该模型如下图所示，该模型中涉及到三个角色：

- ROS Master (管理者)
- Talker (发布者)
- Listener (订阅者)

ROS Master 负责保管 Talker 和 Listener 注册的信息，并匹配话题相同的 Talker 与 Listener，帮助 Talker 与 Listener 建立连接，连接建立后，Talker 可以发布消息，且发布的消息会被 Listener 订阅。



整个流程由以下步骤实现：

0.Talker注册

Talker启动后，会通过RPC在 ROS Master 中注册自身信息，其中包含所发布消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中。

1.Listener注册

Listener启动后，也会通过RPC在 ROS Master 中注册自身信息，包含需要订阅消息的话题名。ROS Master 会将节点的注册信息加入到注册表中。

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配Talker 和 Listener，并通过 RPC 向 Listener 发送 Talker 的 RPC 地址信息。

3.Listener向Talker发送请求

Listener 根据接收到的 RPC 地址，通过 RPC 向 Talker 发送连接请求，传输订阅的话题名称、消息类型以及通信协议(TCP/UDP)。

4.Talker确认请求

Talker 接收到 Listener 的请求后，也是通过 RPC 向 Listener 确认连接信息，并发送自身的 TCP 地址信息。

5.Listener与Talker件里连接

Listener 根据步骤4 返回的消息使用 TCP 与 Talker 建立网络连接。

6.Talker向Listener发送消息

连接建立后，Talker 开始向 Listener 发布消息。

注意1:上述实现流程中，前五步使用的 RPC协议，最后两步使用的是 TCP 协议

注意2: Talker 与 Listener 的启动无先后顺序要求

注意3: Talker 与 Listener 都可以有多个

注意4: Talker 与 Listener 连接建立后，不再需要 ROS Master。也即，即便关闭ROS Master，Talker 与 Listern 照常通信。

2.1.3 话题通信基本操作B(Python)

需求:

编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个:

1. 发布方
2. 接收方
3. 数据(此处为普通文本)

流程:

1. 编写发布方实现;
2. 编写订阅方实现;
3. 为python文件添加可执行权限;
4. 编辑配置文件;
5. 编译并执行。

1.发布方

```

1  #! /usr/bin/env python
2  """
3      需求：实现基本的话题通信，一方发布数据，一方接收数据，
4      实现的关键点：
5          1.发送方
6          2.接收方
7          3.数据(此处为普通文本)
8
9      PS：二者需要设置相同的话题
10
11
12      消息发布方：
13          循环发布信息:HelloWorld 后缀数字编号
14
15      实现流程：
16          1.导包
17          2.初始化 ROS 节点:命名(唯一)
18          3.实例化 发布者 对象
19          4.组织被发布的数据，并编写逻辑发布数据
20
21
22  """
23  #1.导包
24  import rospy
25  from std_msgs.msg import String
26
27  if __name__ == "__main__":
28      #2.初始化 ROS 节点:命名(唯一)
29      rospy.init_node("talker_p")
30      #3.实例化 发布者 对象
31      pub = rospy.Publisher("chatter",String,queue_size=10)
32      #4.组织被发布的数据，并编写逻辑发布数据
33      msg = String() #创建 msg 对象
34      msg_front = "hello 你好"
35      count = 0 #计数器
36      # 设置循环频率
37      rate = rospy.Rate(1)
38      while not rospy.is_shutdown():
39
40          #拼接字符串
41          msg.data = msg_front + str(count)
42
43          pub.publish(msg)
44          rate.sleep()
45          rospy.loginfo("写出的数据:%s",msg.data)
46          count += 1

```

2.订阅方

```

1  #! /usr/bin/env python
2  """

```

```

3      需求：实现基本的话题通信，一方发布数据，一方接收数据，
4      实现的关键点：
5          1.发送方
6          2.接收方
7          3.数据(此处为普通文本)
8
9
10     消息订阅方：
11         订阅话题并打印接收到的消息
12
13     实现流程：
14         1.导包
15         2.初始化 ROS 节点:命名(唯一)
16         3.实例化 订阅者 对象
17         4.处理订阅的消息(回调函数)
18         5.设置循环调用回调函数
19
20
21
22     """
23     #1.导包
24     import rospy
25     from std_msgs.msg import String
26
27     def doMsg(msg):
28         rospy.loginfo("I heard:%s",msg.data)
29
30     if __name__ == "__main__":
31         #2.初始化 ROS 节点:命名(唯一)
32         rospy.init_node("listener_p")
33         #3.实例化 订阅者 对象
34         sub = rospy.Subscriber("chatter",String,doMsg,queue_size=10)
35         #4.处理订阅的消息(回调函数)
36         #5.设置循环调用回调函数
37         rospy.spin()

```

3.添加可执行权限

终端下进入 scripts 执行: `chmod +x *.py`

4.配置 CMakeLists.txt

```

1 catkin_install_python(PROGRAMS
2     scripts/talker_p.py
3     scripts/listener_p.py
4     DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
5 )

```

5.执行

- 1.启动 roscore;
- 2.启动发布节点;
- 3.启动订阅节点。

运行结果与引言部分的演示案例1类似。

PS: 可以使用 `rqt_graph` 查看节点关系。

2.1.4 话题通信自定义msg

在 ROS 通信协议中，数据载体是一个较为重要组成部分，ROS 中通过 `std_msgs` 封装了一些原生的数据类型,比如:String、Int32、Int64、Char、Bool、Empty.... 但是，这些数据一般只包含一个 `data` 字段，结构的单一意味着功能上的局限性，当传输一些复杂的数据，比如: 激光雷达的信息... `std_msgs` 由于描述性较差而显得力不从心，这种场景下可以使用自定义的消息类型

`msgs`只是简单的文本文件，每行具有字段类型和字段名称，可以使用的字段类型有：

- int8, int16, int32, int64 (或者无符号类型: uint*)
- float32, float64
- string
- time, duration
- other msg files
- variable-length array[] and fixed-length array[C]

ROS中还有一种特殊类型：`Header`，标头包含时间戳和ROS中常用的坐标帧信息。会经常看到 `msg`文件的第一行具有 `Header`标头。

需求:创建自定义消息，该消息包含人的信息:姓名、身高、年龄等。

流程:

1. 按照固定格式创建 `msg` 文件
2. 编辑配置文件
3. 编译生成可以被 Python 或 C++ 调用的中间文件

1.定义msg文件

功能包下新建 `msg` 目录，添加文件 `Person.msg`

```
1 | string name
2 | uint16 age
3 | float64 height
```

2.编辑配置文件

`package.xml`中添加编译依赖与执行依赖

```

1 <build_depend>message_generation</build_depend>
2 <exec_depend>message_runtime</exec_depend>
3 <!--
4 exce_depend 以前对应的是 run_depend 现在非法
5 -->

```

CMakeLists.txt编辑 msg 相关配置

```

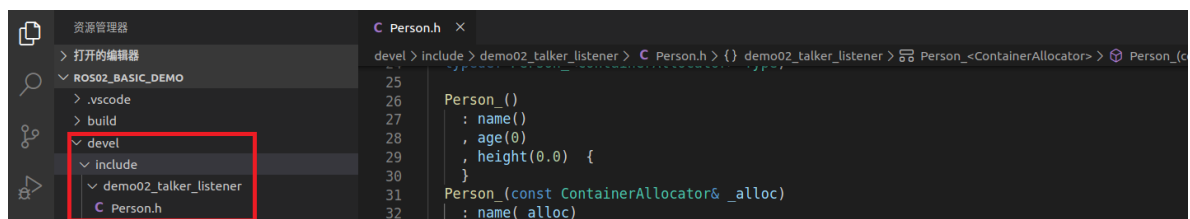
1 find_package(catkin REQUIRED COMPONENTS
2   roscpp
3   rospy
4   std_msgs
5   message_generation
6 )
7 # 需要加入 message_generation,必须有 std_msgs
8
9 ## 配置 msg 源文件
10 add_message_files(
11   FILES
12   Person.msg
13 )
14
15 # 生成消息时依赖于 std_msgs
16 generate_messages(
17   DEPENDENCIES
18   std_msgs
19 )
20
21 #运行时依赖
22 catkin_package(
23   # INCLUDE_DIRS include
24   # LIBRARIES demo02_talker_listener
25   CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
26   # DEPENDS system_lib
27 )

```

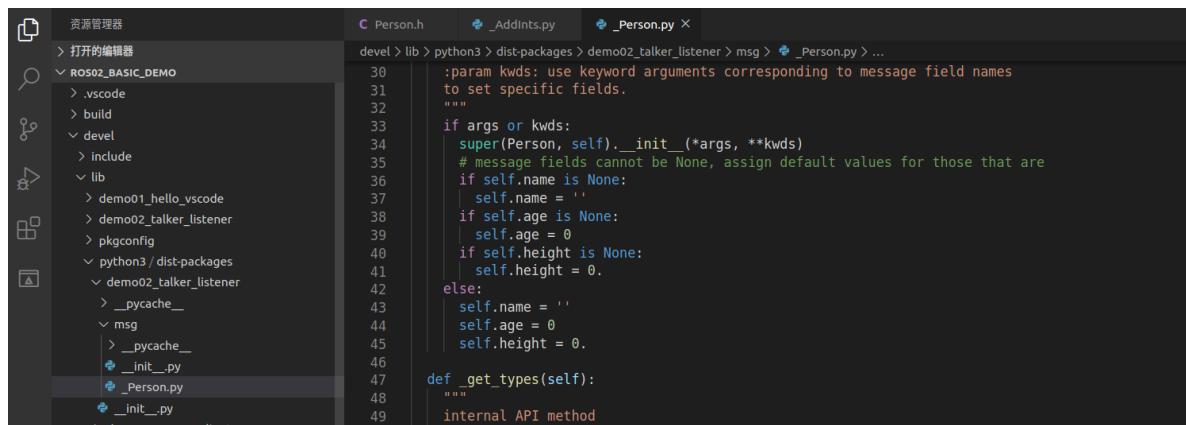
3.编译

编译后的中间文件查看:

C++ 需要调用的中间文件(.../工作空间/devel/include/包名/xxx.h)



Python 需要调用的中间文件(.../工作空间/devel/lib/python3/dist-packages/包名/msg)



后续调用相关 `msg` 时，是从这些中间文件调用的

2.1.6 话题通信自定义msg调用B(Python)

需求:

编写发布订阅实现，要求发布方以1HZ(每秒1次)的频率发布自定义消息，订阅方订阅自定义消息并将消息内容打印输出。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个:

1. 发布方
2. 接收方
3. 数据(此处为自定义消息)

流程:

1. 编写发布方实现;
2. 编写订阅方实现;
3. 为python文件添加可执行权限;
4. 编辑配置文件;
5. 编译并执行。

0.vscode配置

为了方便代码提示以及误抛异常，需要先配置 `vscode`，将前面生成的 `python` 文件路径配置进 `settings.json`

```
1 {
2     "python.autoComplete.extraPaths": [
3         "/opt/ros/noetic/lib/python3/dist-packages",
4         "/xxx/yyy工作空间/devel/lib/python3/dist-packages"
5     ]
6 }
```

1.发布方

```
1  #! /usr/bin/env python
2  """
3      发布方:
4          循环发送消息
5
6  """
7  import rospy
8  from demo02_talker_listener.msg import Person
9
10
11  if __name__ == "__main__":
12      #1.初始化 ROS 节点
13      rospy.init_node("talker_person_p")
14      #2.创建发布者对象
15      pub = rospy.Publisher("chatter_person",Person,queue_size=10)
16      #3.组织消息
17      p = Person()
18      p.name = "葫芦瓦"
19      p.age = 18
20      p.height = 0.75
21
22      #4.编写消息发布逻辑
23      rate = rospy.Rate(1)
24      while not rospy.is_shutdown():
25          pub.publish(p) #发布消息
26          rate.sleep() #休眠
27          rospy.loginfo("姓名:%s, 年龄:%d, 身高:%.2f",p.name, p.age, p.height)
```

2.订阅方

```
1  #! /usr/bin/env python
2  """
3      订阅方:
4          订阅消息
5
6  """
7  import rospy
8  from demo02_talker_listener.msg import Person
9
10  def doPerson(p):
11      rospy.loginfo("接收到的人的信息:%s, %d, %.2f",p.name, p.age, p.height)
12
13
14  if __name__ == "__main__":
15      #1.初始化节点
16      rospy.init_node("listener_person_p")
17      #2.创建订阅者对象
18      sub = rospy.Subscriber("chatter_person",Person,doPerson,queue_size=10)
19      rospy.spin() #4.循环
```

3.权限设置

终端下进入 scripts 执行: `chmod +x *.py`

4.配置 CMakeLists.txt

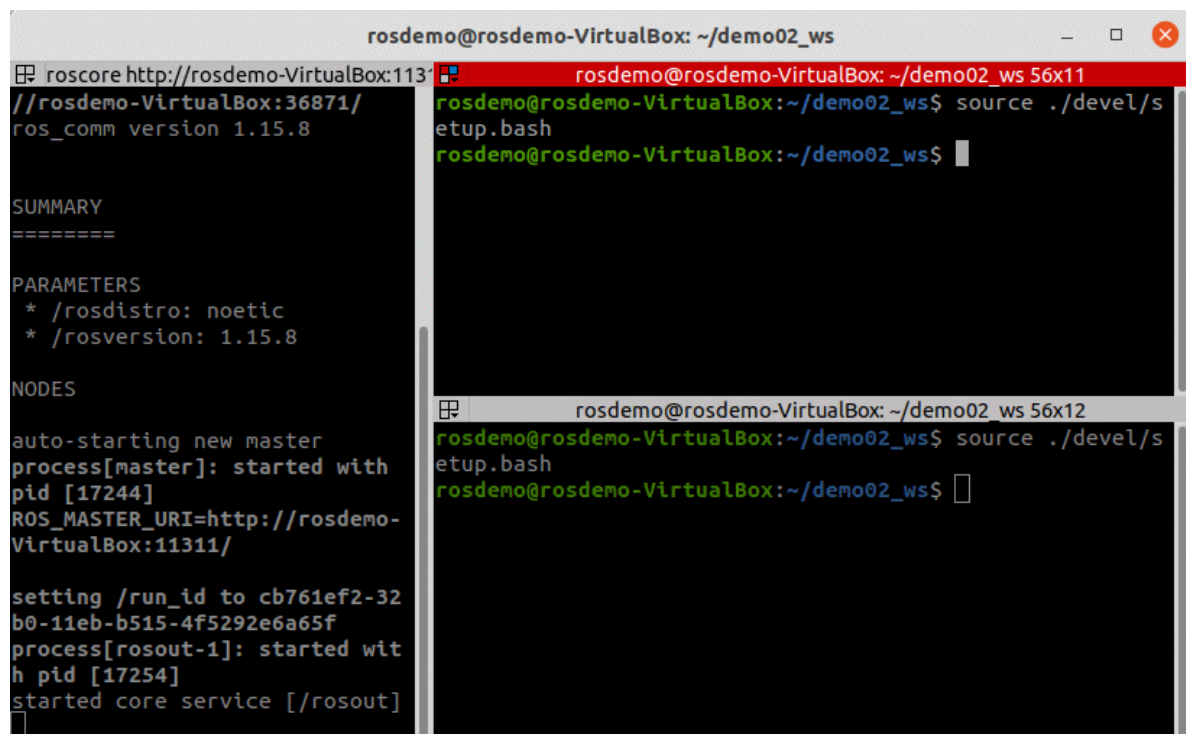
```
1 catkin_install_python(PROGRAMS
2   scripts/talker_p.py
3   scripts/listener_p.py
4   scripts/person_talker.py
5   scripts/person_listener.py
6   DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
7 )
```

5.执行

- 1.启动 roscore;
- 2.启动发布节点;
- 3.启动订阅节点。

运行结果与引言部分的演示案例2类似。

PS: 可以使用 `rqt_graph` 查看节点关系。



The image shows two terminal windows from a ROS environment. The left window, titled 'rostdemo@rostdemo-VirtualBox: ~/demo02_ws', displays the output of 'roscore http://rostdemo-VirtualBox:11311/'. It shows the ROS version (1.15.8), a summary, parameters for 'roscore' (noetic, 1.15.8), and a list of nodes including 'auto-starting new master', 'process[roscore]: started with pid [17244]', 'ROS_MASTER_URI=http://rostdemo-VirtualBox:11311/', 'setting /run_id to cb761ef2-32b0-11eb-b515-4f5292e6a65f', 'process[roscore-1]: started with pid [17254]', and 'started core service [/roscore]'.

The right window, titled 'rostdemo@rostdemo-VirtualBox: ~/demo02_ws 56x11', shows the command 'source ./devel/setup.bash' being executed, followed by a prompt 'rostdemo@rostdemo-VirtualBox:~/demo02_ws\$'.

The bottom window, titled 'rostdemo@rostdemo-VirtualBox: ~/demo02_ws 56x12', shows the command 'source ./devel/setup.bash' being executed, followed by a prompt 'rostdemo@rostdemo-VirtualBox:~/demo02_ws\$'.

2.2 服务通信

服务通信也是ROS中一种极其常用的通信模式，服务通信是基于请求响应模式的，是一种应答机制。也即: 一个节点A向另一个节点B发送请求，B接收处理请求并产生响应结果返回给A。比如如下场景:

机器人巡逻过程中，控制系统分析传感器数据发现可疑物体或人... 此时需要拍摄照片并留存。

在上述场景中，就使用到了服务通信。

- 一个节点需要向相机节点发送拍照请求，相机节点处理请求，并返回处理结果

与上述应用类似的，服务通信更适用于对时时性有要求、具有一定逻辑处理的应用场景。

概念

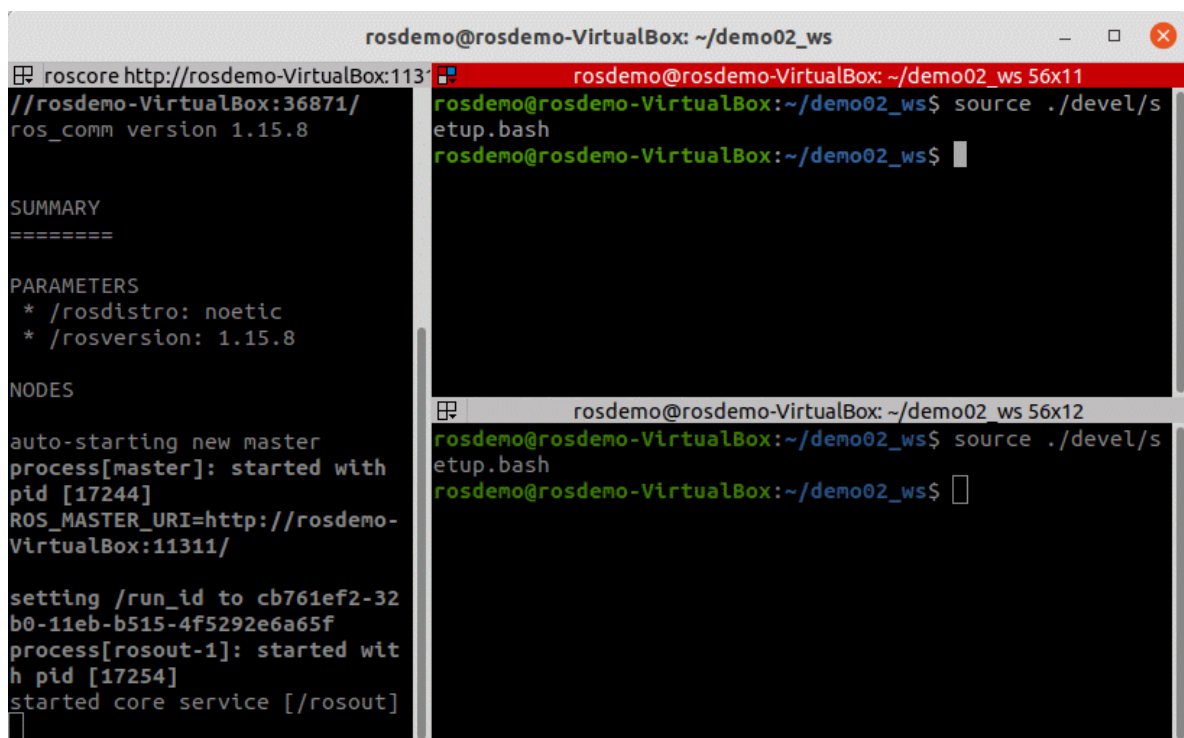
以请求响应的方式实现不同节点之间数据交互的通信模式。

作用

用于偶然的、对时时性有要求、有一定逻辑处理需求的数据传输场景。

案例

实现两个数字的求和，客户端节点，运行会向服务器发送两个数字，服务器端节点接收两个数字求和并将结果响应回客户端。



```
roscat http://roscat-VirtualBox:11311/
//roscat-VirtualBox:36871/
ros_comm version 1.15.8

SUMMARY
=====

PARAMETERS
* /roscat: noetic
* /roscat: 1.15.8

NODES
auto-starting new master
process[roscat]: started with
pid [17244]
ROS_MASTER_URI=http://roscat-
VirtualBox:11311/

setting /run_id to cb761ef2-32
b0-11eb-b515-4f5292e6a65f
process[roscat-1]: started wit
h pid [17254]
started core service [/roscat]

```

```
roscat@roscat-VirtualBox: ~/demo02_ws 56x11
roscat@roscat-VirtualBox:~/demo02_ws$ source ./devel/s
etup.bash
roscat@roscat-VirtualBox:~/demo02_ws$

roscat@roscat-VirtualBox: ~/demo02_ws 56x12
roscat@roscat-VirtualBox:~/demo02_ws$ source ./devel/s
etup.bash
roscat@roscat-VirtualBox:~/demo02_ws$
```

另请参考:

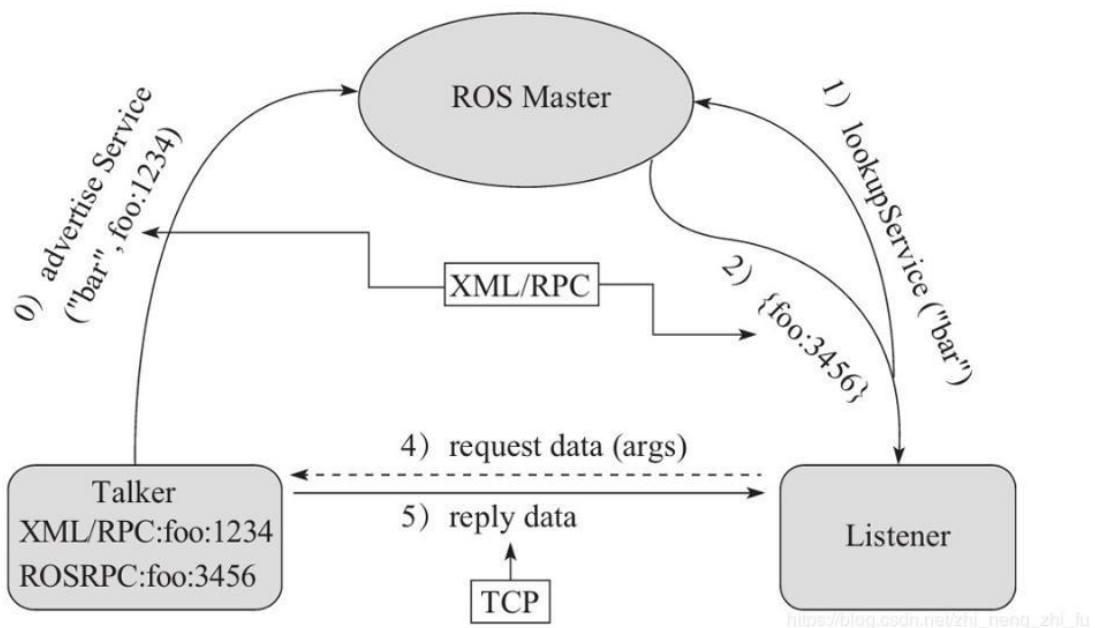
- <http://wiki.ros.org/ROS/Tutorials/CreatingMsgAndSrv>
- <http://wiki.ros.org/ROS/Tutorials/WritingServiceClient%28c%2B%2B%29>
- <http://wiki.ros.org/ROS/Tutorials/WritingServiceClient%28python%29>

2.2.1 服务通信理论模型

服务通信较之于话题通信更简单些，理论模型如下图所示，该模型中涉及到三个角色:

- ROS master(管理者)
- Server(服务端)
- Client(客户端)

ROS Master 负责保管 Server 和 Client 注册的信息，并匹配话题相同的 Server 与 Client，帮助 Server 与 Client 建立连接，连接建立后，Client 发送请求信息，Server 返回响应信息。



整个流程由以下步骤实现:

0.Server注册

Server 启动后，会通过RPC在 ROS Master 中注册自身信息，其中包含提供的服务的名称。ROS Master 会将节点的注册信息加入到注册表中。

1.Client注册

Client 启动后，也会通过RPC在 ROS Master 中注册自身信息，包含需要请求的服务的名称。ROS Master 会将节点的注册信息加入到注册表中。

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配Server和 Client，并通过 RPC 向 Client 发送 Server 的 **TCP** 地址信息。

3.Client发送请求

Client 根据步骤2响应的信息，使用 TCP 与 Server 建立网络连接，并发送请求数据。

4.Server发送响应

Server 接收、解析请求的数据，并产生响应结果返回给 Client。

注意:

- 1.客户端请求被处理时，需要保证服务器已经启动；
- 2.服务端和客户端都可以存在多个。

2.2.2 服务通信自定义srv

需求:

服务通信中，客户端提交两个整数至服务端，服务端求和并响应结果到客户端，请创建服务器与客户端通信的数据载体。

流程:

srv 文件内的可用数据类型与 msg 文件一致，且定义 srv 实现流程与自定义 msg 实现流程类似:

1. 按照固定格式创建srv文件
2. 编辑配置文件
3. 编译生成中间文件

1.定义srv文件

服务通信中，数据分成两部分，请求与响应，在 srv 文件中请求和响应使用 `---` 分割，具体实现如下:

功能包下新建 srv 目录，添加 xxx.srv 文件，内容:

```
1  # 客户端请求时发送的两个数字
2  int32 num1
3  int32 num2
4  ---
5  # 服务器响应发送的数据
6  int32 sum
```

2.编辑配置文件

package.xml中添加编译依赖与执行依赖

```
1  <build_depend>message_generation</build_depend>
2  <exec_depend>message_runtime</exec_depend>
3  <!--
4  exce_depend 以前对应的是 run_depend 现在非法
5  -->
```

CMakeLists.txt编辑 srv 相关配置

```
1  find_package(catkin REQUIRED COMPONENTS
2    roscpp
3    rospy
```

```

4     std_msgs
5     message_generation
6 )
7 # 需要加入 message_generation,必须有 std_msgs
8
9 add_service_files(
10     FILES
11     AddInts.srv
12 )
13
14 generate_messages(
15     DEPENDENCIES
16     std_msgs
17 )

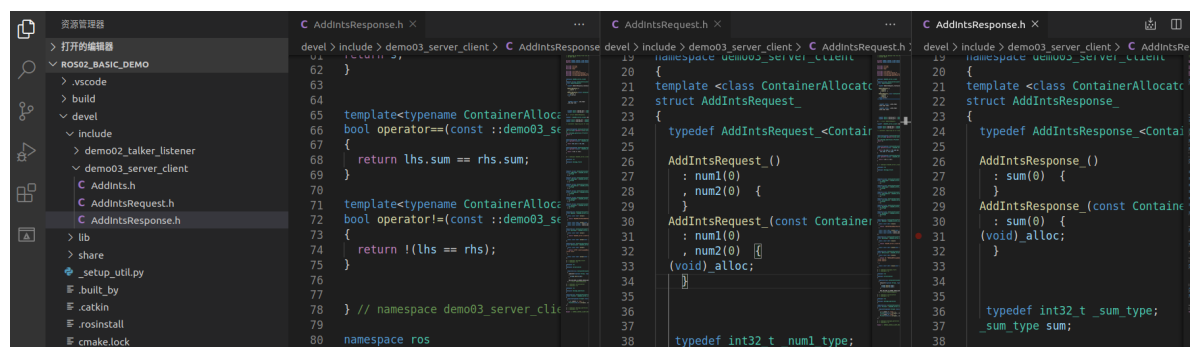
```

注意: 官网没有在 catkin_package 中配置 message_runtime,经测试配置也可以

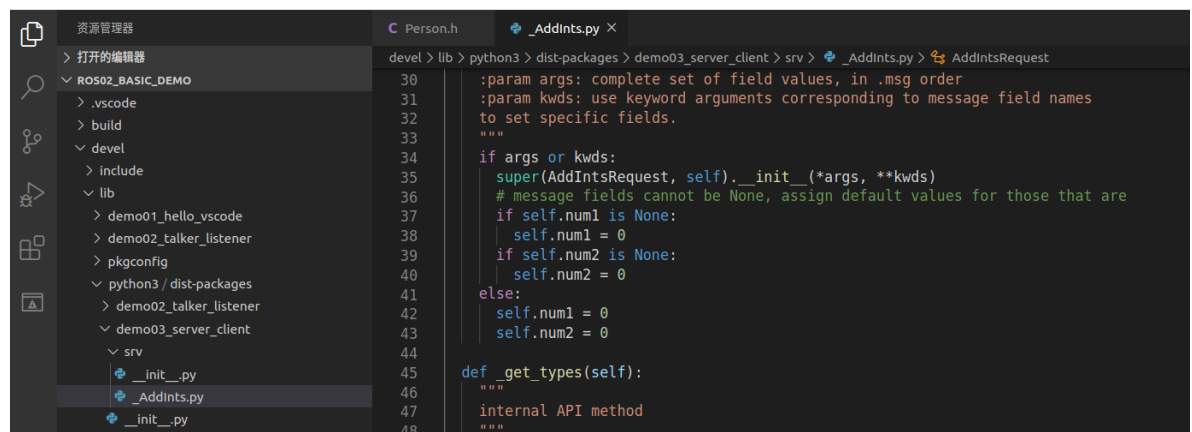
3.编译

编译后的中间文件查看:

C++ 需要调用的中间文件(.../工作空间/devel/include/包名/xxx.h)



Python 需要调用的中间文件(.../工作空间/devel/lib/python3/dist-packages/包名/srv)



后续调用相关 srv 时, 是从这些中间文件调用的

2.2.4 服务通信自定义srv调用B(Python)

需求:

编写服务通信，客户端提交两个整数至服务端，服务端求和并响应结果到客户端。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个:

1. 服务端
2. 客户端
3. 数据

流程:

1. 编写服务端实现;
2. 编写客户端实现;
3. 为python文件添加可执行权限;
4. 编辑配置文件;
5. 编译并执行。

0.vscode配置

需要像之前自定义 msg 实现一样配置settings.json 文件，如果以前已经配置且没有变更工作空间，可以忽略，如果需要配置，配置方式与之前相同:

```
1 {  
2     "python.autoComplete.extraPaths": [  
3         "/opt/ros/noetic/lib/python3/dist-packages",  
4     ]  
5 }
```

1.服务端

```
1  #!/usr/bin/env python  
2  """  
3      需求:  
4          编写两个节点实现服务通信，客户端节点需要提交两个整数到服务器  
5          服务器需要解析客户端提交的数据，相加后，将结果响应回客户端，  
6          客户端再解析  
7  
8      服务器端实现:  
9          1. 导包  
10         2. 初始化 ROS 节点  
11         3. 创建服务对象  
12         4. 回调函数处理请求并产生响应  
13         5. spin 函数  
14  
15     """  
16     # 1. 导包  
17     import rospy  
18     from demo03_server_client.srv import AddInts, AddIntsRequest, AddIntsResponse
```

```

19 # 回调函数的参数是请求对象，返回值是响应对象
20 def doReq(req):
21     # 解析提交的数据
22     sum = req.num1 + req.num2
23     rospy.loginfo("提交的数据:num1 = %d, num2 = %d, sum = %d",req.num1,
24 req.num2, sum)
25
26     # 创建响应对象，赋值并返回
27     # resp = AddIntsResponse()
28     # resp.sum = sum
29     resp = AddIntsResponse(sum)
30     return resp
31
32 if __name__ == "__main__":
33     # 2.初始化 ROS 节点
34     rospy.init_node("addints_server_p")
35     # 3.创建服务对象
36     server = rospy.Service("AddInts",AddInts,doReq)
37     # 4.回调函数处理请求并产生响应
38     # 5.spin 函数
39     rospy.spin()

```

2.客户端

```

1  #!/usr/bin/env python
2
3  """
4      需求:
5          编写两个节点实现服务通信，客户端节点需要提交两个整数到服务器
6          服务器需要解析客户端提交的数据，相加后，将结果响应回客户端，
7          客户端再解析
8
9      客户端实现:
10         1.导包
11         2.初始化 ROS 节点
12         3.创建请求对象
13         4.发送请求
14         5.接收并处理响应
15
16     优化:
17         加入数据的动态获取
18
19 """
20
21 #1.导包
22 import rospy
23 from demo03_server_client.srv import *
24 import sys
25
26 if __name__ == "__main__":

```

```

27
28     #优化实现
29     if len(sys.argv) != 3:
30         rospy.logerr("请正确提交参数")
31         sys.exit(1)
32
33
34     # 2.初始化 ROS 节点
35     rospy.init_node("AddInts_Client_p")
36     # 3.创建请求对象
37     client = rospy.ServiceProxy("AddInts",AddInts)
38     # 请求前, 等待服务已经就绪
39     # 方式1:
40     # rospy.wait_for_service("AddInts")
41     # 方式2
42     client.wait_for_service()
43     # 4.发送请求,接收并处理响应
44     # 方式1
45     # resp = client(3,4)
46     # 方式2
47     # resp = client(AddIntsRequest(1,5))
48     # 方式3
49     req = AddIntsRequest()
50     # req.num1 = 100
51     # req.num2 = 200
52
53     #优化
54     req.num1 = int(sys.argv[1])
55     req.num2 = int(sys.argv[2])
56
57     resp = client.call(req)
58     rospy.loginfo("响应结果:%d",resp.sum)

```

3.设置权限

终端下进入 scripts 执行: `chmod +x *.py`

4.配置 CMakeLists.txt

CMakeLists.txt

```

1 catkin_install_python(PROGRAMS
2   scripts/AddInts_Server_p.py
3   scripts/AddInts_Client_p.py
4   DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
5 )

```

5.执行

流程:

- 需要先启动服务: `roslaunch 包名 服务`
- 然后再调用客户端: `roscall 包名 客户端 参数1 参数2`

结果:

会根据提交的数据响应相加后的结果。

2.3 参数服务器

参数服务器在ROS中主要用于实现不同节点之间的数据共享。参数服务器相当于独立于所有节点的一个公共容器,可以将数据存储在该容器中,被不同的节点调用,当然不同的节点也可以往其中存储数据,关于参数服务器的典型应用场景如下:

导航实现时,会进行路径规划,比如:全局路径规划,设计一个从出发点到目标点的大致路径。本地路径规划,会根据当前路况生成时时的行进路径

上述场景中,全局路径规划和本地路径规划时,就会使用到参数服务器:

- 路径规划时,需要参考小车的尺寸,我们可以将这些尺寸信息存储到参数服务器,全局路径规划节点与本地路径规划节点都可以从参数服务器中调用这些参数

参数服务器,一般适用于存在数据共享的一些应用场景。

概念

以共享的方式实现不同节点之间数据交互的通信模式。

作用

存储一些多节点共享的数据,类似于全局变量。

案例

实现参数增删改查操作。

另请参考:

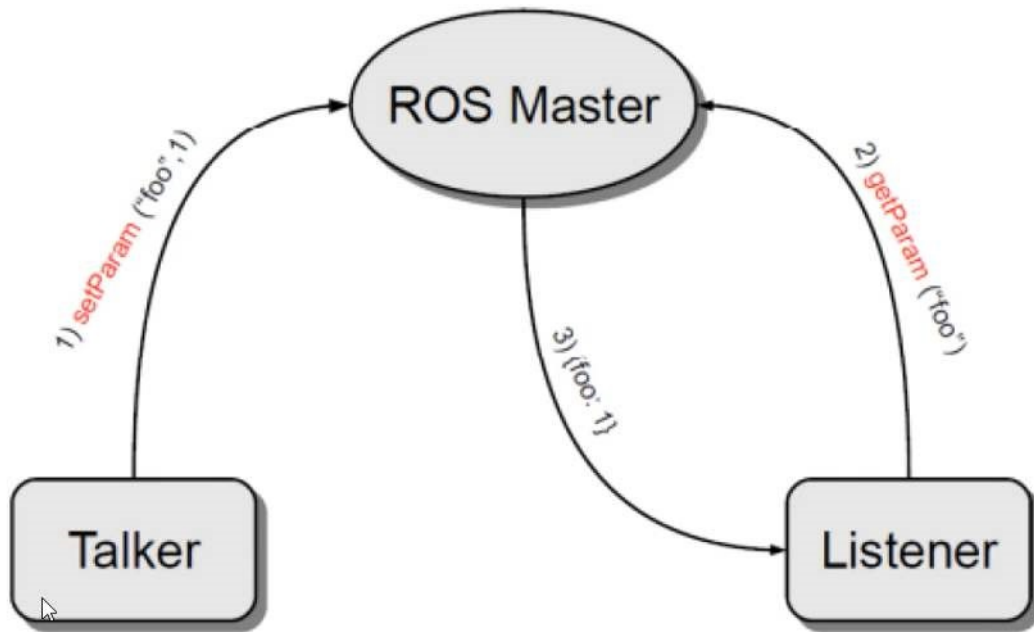
- <http://wiki.ros.org/Parameter%20Server>
- <http://wiki.ros.org/roscpp/Overview/Parameter%20Server>
- <http://wiki.ros.org/rospy/Overview/Parameter%20Server>

2.3.1 参数服务器理论模型

参数服务器实现是最为简单的,该模型如下图所示,该模型中涉及到三个角色:

- ROS Master (管理者)
- Talker (参数设置者)
- Listener (参数调用者)

ROS Master 作为一个公共容器保存参数，Talker 可以向容器中设置参数，Listener 可以获取参数。



整个流程由以下步骤实现:

1.Talker 设置参数

Talker 通过 RPC 向参数服务器发送参数(包括参数名与参数值)，ROS Master 将参数保存到参数列表中。

2.Listener 获取参数

Listener 通过 RPC 向参数服务器发送参数查找请求，请求中包含要查找的参数名。

3.ROS Master 向 Listener 发送参数值

ROS Master 根据步骤2请求提供的参数名查找参数值，并将查询结果通过 RPC 发送给 Listener。

参数可使用数据类型:

- 32-bit integers
- booleans
- strings
- doubles
- iso8601 dates
- lists
- base64-encoded binary data
- 字典

注意:参数服务器不是为高性能而设计的，因此最好用于存储静态的非二进制的简单数据

2.3.3 参数操作B(Python)

需求:实现参数服务器参数的增删改查操作。

1.参数服务器新增(修改)参数

```
1  #! /usr/bin/env python
2  """
3      参数服务器操作之新增与修改(二者API一样)_Python实现:
4  """
5
6  import rospy
7
8  if __name__ == "__main__":
9      rospy.init_node("set_update_paramter_p")
10
11     # 设置各种类型参数
12     rospy.set_param("p_int",10)
13     rospy.set_param("p_double",3.14)
14     rospy.set_param("p_bool",True)
15     rospy.set_param("p_string","hello python")
16     rospy.set_param("p_list",["hello","haha","xixi"])
17     rospy.set_param("p_dict",{"name":"hulu","age":8})
18
19     # 修改
20     rospy.set_param("p_int",100)
```

2.参数服务器获取参数

```
1  #! /usr/bin/env python
2
3  """
4      参数服务器操作之查询_Python实现:
5          get_param(键,默认值)
6              当键存在时, 返回对应的值, 如果不存在返回默认值
7          get_param_cached
8          get_param_names
9          has_param
10         search_param
11  """
12
13  import rospy
14
15  if __name__ == "__main__":
16      rospy.init_node("get_param_p")
17
18      #获取参数
19      int_value = rospy.get_param("p_int",10000)
20      double_value = rospy.get_param("p_double")
21      bool_value = rospy.get_param("p_bool")
22      string_value = rospy.get_param("p_string")
```

```

23     p_list = rospy.get_param("p_list")
24     p_dict = rospy.get_param("p_dict")
25
26     rospy.loginfo("获取的数据:%d,%.2f,%d,%s",
27                   int_value,
28                   double_value,
29                   bool_value,
30                   string_value)
31     for ele in p_list:
32         rospy.loginfo("ele = %s", ele)
33
34     rospy.loginfo("name = %s, age = %d",p_dict["name"],p_dict["age"])
35
36     # get_param_cached
37     int_cached = rospy.get_param_cached("p_int")
38     rospy.loginfo("缓存数据:%d",int_cached)
39
40     # get_param_names
41     names = rospy.get_param_names()
42     for name in names:
43         rospy.loginfo("name = %s",name)
44
45     rospy.loginfo("-"*80)
46
47     # has_param
48     flag = rospy.has_param("p_int")
49     rospy.loginfo("包含p_int吗? %d",flag)
50
51     # search_param
52     key = rospy.search_param("p_int")
53     rospy.loginfo("搜索的键 = %s",key)

```

3.参数服务器删除参数

```

1  #! /usr/bin/env python
2  """
3      参数服务器操作之删除_Python实现:
4      rospy.delete_param("键")
5      键存在时, 可以删除成功, 键不存在时, 会抛出异常
6  """
7  import rospy
8
9  if __name__ == "__main__":
10     rospy.init_node("delete_param_p")
11
12     try:
13         rospy.delete_param("p_int")
14     except Exception as e:
15         rospy.loginfo("删除失败")

```

2.4 常用命令

机器人系统中启动的节点少则几个，多则十几个、几十个，不同的节点名称各异，通信时使用话题、服务、消息、参数等等都各不相同，一个显而易见的问题是：当需要自定义节点和其他某个已经存在的节点通信时，如何获取对方的话题、以及消息载体的格式呢？

在 ROS 同提供了一些实用的命令行工具，可以用于获取不同节点的各类信息，常用的命令如下：

- `roscall` : 操作节点
- `rostopic` : 操作话题
- `rosservice` : 操作服务
- `rosmmsg` : 操作msg消息
- `rossrv` : 操作srv消息
- `rosparam` : 操作参数

作用

和之前介绍的文件系统操作命令比较，文件操作命令是静态的，操作的是磁盘上的文件，而上述命令是动态的，在ROS程序启动后，可以动态的获取运行中的节点或参数的相关信息。

案例

本节将借助于2.1、2.2和2.3的通信实现介绍相关命令的基本使用，并通过练习ROS内置的小海龟例程来强化命令的应用。

另请参考：

- <http://wiki.ros.org/ROS/CommandLineTools>

ROS Command-line tools

目录

1. ROS Command-line tools
 1. Common user tools
 1. [roscd](#)
 2. [rosclean](#)
 3. [roscd](#)
 4. [rosclean](#)
 5. [roscd](#)
 6. [roscd](#)
 7. [roscd](#)
 8. [roscd](#)
 9. [roscd](#)
 10. [roscd](#)
 11. [roscd](#)
 12. [roscd](#)
 13. [roscd](#)
 14. [roscd](#)
 15. [roscd](#)
 16. [roscd](#)
 17. [roscd](#)
 18. [roscd](#)
 19. [roscd](#)
 20. [roscd](#)
 21. [roscd](#)
 22. [roscd](#)
 2. Graphical tools
 1. [roscd](#)
 2. [roscd](#)
 3. [roscd](#)
 4. [roscd](#)
 3. Less-used tools
 1. [roscd](#)

维基

- [Distributions](#)
- [ROS/Installation](#)
- [ROS/Tutorials](#)
- [RecentChanges](#)
- [ROS/CommandLineTools](#)

网页

[只读网页](#)

[信息](#)

[附件](#)

更多操作:

[源码](#)

[执行](#)

用户

[登录](#)

2.4.1 roscd

roscd 是用于获取节点信息的命令

- | | | |
|---|----------------------------|------------|
| 1 | <code>roscd ping</code> | 测试到节点的连接状态 |
| 2 | <code>roscd list</code> | 列出活动节点 |
| 3 | <code>roscd info</code> | 打印节点信息 |
| 4 | <code>roscd machine</code> | 列出指定设备上节点 |
| 5 | <code>roscd kill</code> | 杀死某个节点 |
| 6 | <code>roscd cleanup</code> | 清除不可连接的节点 |

- roscd ping
测试到节点的连接状态
- roscd list
列出活动节点
- roscd info
打印节点信息
- roscd machine
列出指定设备上的节点
- roscd kill
杀死某个节点
- roscd cleanup

清除无用节点，启动乌龟节点，然后 `ctrl + c` 关闭，该节点并没被彻底清除，可以使用 `cleanup` 清除节点

2.4.2 rostopic

rostopic包含rostopic命令行工具，用于显示有关ROS 主题的调试信息，包括发布者，订阅者，发布频率和ROS消息。它还包含一个实验性Python库，用于动态获取有关主题的信息并与之交互。

```
1 | rostopic bw      显示主题使用的带宽
2 | rostopic delay   显示带有 header 的主题延迟
3 | rostopic echo    打印消息到屏幕
4 | rostopic find    根据类型查找主题
5 | rostopic hz      显示主题的发布频率
6 | rostopic info    显示主题相关信息
7 | rostopic list    显示所有活动状态下的主题
8 | rostopic pub     将数据发布到主题
9 | rostopic type    打印主题类型
10 | Copy
```

- **rostopic list(-v)**

直接调用即可，控制台将打印当前运行状态下的主题名称

`rostopic list -v`: 获取话题详情(比如列出：发布者和订阅者个数...)

- **rostopic pub**

可以直接调用命令向订阅者发布消息

为roboware 自动生成的 发布/订阅 模型案例中的 订阅者 发布一条字符串

```
1 | rostopic pub /主题名称 消息类型 消息内容
2 | rostopic pub /chatter std_msgs gagaxixi
```

为 小乌龟案例的 订阅者 发布一条运动信息

```
1 | rostopic pub /turtle1/cmd_vel geometry_msgs/Twist
2 |   "linear:
3 |     x: 1.0
4 |     y: 0.0
5 |     z: 0.0
6 |   angular:
7 |     x: 0.0
8 |     y: 0.0
9 |     z: 2.0"
10 | //只发布一次运动信息
11 |
12 | rostopic pub -r 10 /turtle1/cmd_vel geometry_msgs/Twist
13 |   "linear:
14 |     x: 1.0
15 |     y: 0.0
16 |     z: 0.0
17 |   angular:
```

```

18 | x: 0.0
19 | y: 0.0
20 | z: 2.0"
21 | // 以 10HZ 的频率循环发送运动信息

```

- **rostopic echo**

获取指定话题当前发布的消息

- **rostopic info**

获取当前话题的小关信息

消息类型

发布者信息

订阅者信息

- **rostopic type**

列出话题的消息类型

- **rostopic find 消息类型**

根据消息类型查找话题

- **rostopic delay**

列出消息头信息

- **rostopic hz**

列出消息发布频率

- **rostopic bw**

列出消息发布带宽

2.4.3 rosmmsg

rosmmsg是用于显示有关 ROS消息类型的 信息的命令行工具。

rosmmsg 演示

```

1 | rosmmsg show    显示消息描述
2 | rosmmsg info    显示消息信息
3 | rosmmsg list    列出所有消息
4 | rosmmsg md5     显示 md5 加密后的消息
5 | rosmmsg package 显示某个功能包下的所有消息
6 | rosmmsg packages 列出包含消息的功能包

```

- **rosmmsg list**

会列出当前 ROS 中的所有 msg

- **rosmmsg packages**

列出包含消息的所有包

- **rosmmsg package**

列出某个包下的所有msg

```
1 //rosmg package 包名
2 rosmg package turtlesim
```

- rosmg show

显示消息描述

```
1 //rosmg show 消息名称
2 rosmg show turtlesim/Pose
3 结果:
4 float32 x
5 float32 y
6 float32 theta
7 float32 linear_velocity
8 float32 angular_velocity
```

- rosmg info

作用与 rosmg show 一样

- rosmg md5 (资

料:http://wiki.ros.org/ROS/Technical%20Overview#Message_serialization_and_msg_MD5_sums)

一种校验算法，保证数据传输的一致性

2.4.4 rosservice

rosservice包含用于列出和查询ROS **Services** 的rosservice命令行工具。

调用部分服务时，如果对相关工作空间没有配置 path，需要进入工作空间调用 source ./devel/setup.bash

```
1 rosservice args 打印服务参数
2 rosservice call 使用提供的参数调用服务
3 rosservice find 按照服务类型查找服务
4 rosservice info 打印有关服务的信息
5 rosservice list 列出所有活动的服务
6 rosservice type 打印服务类型
7 rosservice uri 打印服务的 ROSRPC uri
```

- rosservice list

列出所有活动的 service

```
1 ~ rosservice list
2 /clear
3 /kill
4 /listener/get_loggers
5 /listener/set_logger_level
6 /reset
7 /rosout/get_loggers
8 /rosout/set_logger_level
9 /rostopic_4985_1578723066421/get_loggers
```

```

10 /rostopic_4985_1578723066421/set_logger_level
11 /rostopic_5582_1578724343069/get_loggers
12 /rostopic_5582_1578724343069/set_logger_level
13 /spawn
14 /turtle1/set_pen
15 /turtle1/teleport_absolute
16 /turtle1/teleport_relative
17 /turtlesim/get_loggers
18 /turtlesim/set_logger_level

```

- rosservice args

打印服务参数

```

1 rosservice args /spawn
2 x y theta name

```

- rosservice call

调用服务

为小乌龟的案例生成一只新的乌龟

```

1 rosservice call /spawn "x: 1.0
2 y: 2.0
3 theta: 0.0
4 name: 'xxx'"
5 name: "xxx"
6
7 //生成一只叫 xxx 的乌龟

```

- rosservice find

根据消息类型获取话题

- rosservice info

获取服务话题详情

- rosservice type

获取消息类型

- rosservice uri

获取服务器 uri

2.4.5 rossrv

rossrv是用于显示有关ROS服务类型的信息的命令行工具，与 rosmmsg 使用语法高度雷同。

```

1 rossrv show      显示服务消息详情
2 rossrv info      显示服务消息相关信息
3 rossrv list      列出所有服务信息
4 rossrv md5       显示 md5 加密后的服务消息
5 rossrv package   显示某个包下所有服务消息
6 rossrv packages  显示包含服务消息的所有包

```

- rossrv list

会列出当前 ROS 中的所有 srv 消息

- rossrv packages

列出包含服务消息的所有包

- rossrv package

列出某个包下的所有msg

```
1 //rossrv package 包名
2 rossrv package turtlesim
```

- rossrv show

显示消息描述

```
1 //rossrv show 消息名称
2 rossrv show turtlesim/Spawn
3 结果:
4 float32 x
5 float32 y
6 float32 theta
7 string name
8 ---
9 string name
```

- rossrv info

作用与 rossrv show 一致

- rossrv md5

对 service 数据使用 md5 校验(加密)

2.4.6 rosparam

rosparam包含rosparam命令行工具，用于使用YAML编码文件在参数服务器上获取和设置ROS参数。

```
1 rosparam set    设置参数
2 rosparam get    获取参数
3 rosparam load   从外部文件加载参数
4 rosparam dump   将参数写出到外部文件
5 rosparam delete 删除参数
6 rosparam list   列出所有参数
```

- rosparam list

列出所有参数

```
1 | rosparam list
2
3 | //默认结果
4 | /roscdistro
5 | /roslaunch/uris/host_helloros_virtual_machine__42911
6 | /rosversion
7 | /run_id
```

- rosparam set

设置参数

```
1 | rosparam set name huluwa
2
3 | //再次调用 rosparam list 结果
4 | /name
5 | /roscdistro
6 | /roslaunch/uris/host_helloros_virtual_machine__42911
7 | /rosversion
8 | /run_id
```

- rosparam get

获取参数

```
1 | rosparam get name
2
3 | //结果
4 | huluwa
```

- rosparam delete

删除参数

```
1 | rosparam delete name
2
3 | //结果
4 | //去除了name
```

- rosparam load(先准备 yaml 文件)

从外部文件加载参数

```
1 | rosparam load xxx.yaml
```

- rosparam dump

将参数写出到外部文件

```
1 | rosparam dump yyy.yaml
```

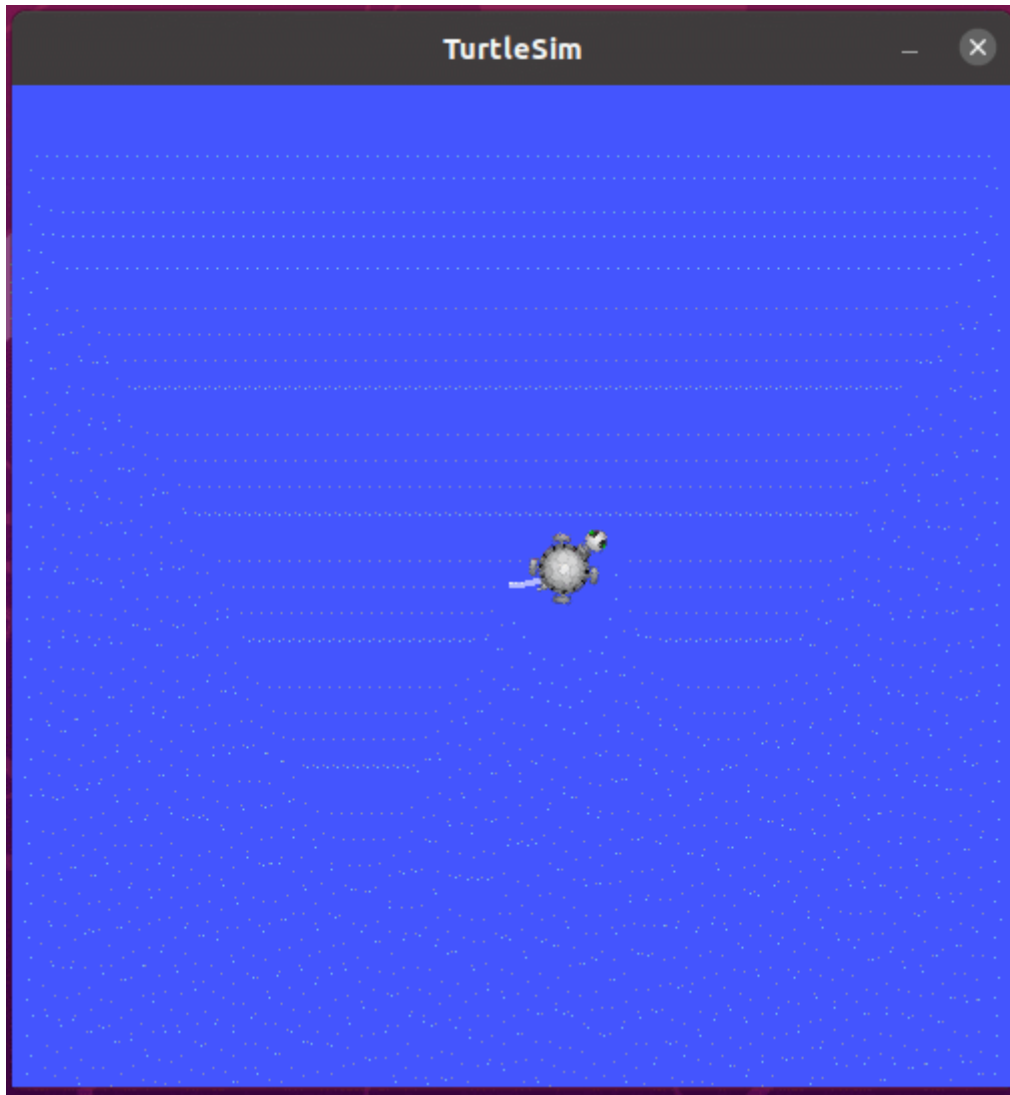
本节主要是通过ROS内置的turtlesim案例，结合已经介绍ROS命令获取节点、话题、话题消息、服务、服务消息与参数的信息，最终再以编码的方式实现乌龟运动的控制、乌龟位姿的订阅、乌龟生成与乌龟窗体背景颜色的修改。

目的:熟悉、强化通信模式应用

5.1 实操01_话题发布

需求描述:编码实现乌龟运动控制，让小乌龟做圆周运动。

结果演示:



实现分析:

1. 乌龟运动控制实现，关键节点有两个，一个是乌龟运动显示节点 `turtlesim_node`，另一个是控制节点，二者是订阅发布模式实现通信的，乌龟运动显示节点直接调用即可，运动控制节点之前是使用的 `turtle_teleop_key`通过键盘 控制，现在需要自定义控制节点。
2. 控制节点自实现时，首先需要了解控制节点与显示节点通信使用的话题与消息，可以使用 `ros`命令结合计算图来获取。
3. 了解了话题与消息之后，通过 C++ 或 Python 编写运动控制节点，通过指定的话题，按照一定的逻辑发布消息即可。

实现流程:

1. 通过计算图结合ros命令获取话题与消息信息。
2. 编码实现运动控制节点。
3. 启动 roscore、turtlesim_node 以及自定义的控制节点，查看运行结果。

1.话题与消息获取

准备: 先启动键盘控制乌龟运动案例。

1.1话题获取

获取话题:/turtle1/cmd_vel

通过计算图查看话题，启动计算图:

```
1 | rqt_graph
```

或者通过 rostopic 列出话题:

```
1 | rostopic list
```

1.2消息获取

获取消息类型:geometry_msgs/Twist

```
1 | rostopic type /turtle1/cmd_vel
```

获取消息格式:

```
1 | rosmmsg info geometry_msgs/Twist
```

响应结果:

```
1 | geometry_msgs/Vector3 linear
2 |   float64 x
3 |   float64 y
4 |   float64 z
5 | geometry_msgs/Vector3 angular
6 |   float64 x
7 |   float64 y
8 |   float64 z
```

linear(线速度) 下的xyz分别对应在x、y和z方向上的速度(单位是 m/s);

angular(角速度)下的xyz分别对应x轴上的翻滚、y轴上俯仰和z轴上偏航的速度(单位是rad/s)。

详情请查看补充资料。

2.实现发布节点

创建功能包需要依赖的功能包: roscpp rospy std_msgs geometry_msgs

实现方案A: C++

```
1  /*
2      编写 ROS 节点, 控制小乌龟画圆
3
4      准备工作:
5          1.获取topic(已知: /turtle1/cmd_vel)
6          2.获取消息类型(已知: geometry_msgs/Twist)
7          3.运行前, 注意先启动 turtlesim_node 节点
8
9      实现流程:
10         1.包含头文件
11         2.初始化 ROS 节点
12         3.创建发布者对象
13         4.循环发布运动控制消息
14  */
15
16  #include "ros/ros.h"
17  #include "geometry_msgs/Twist.h"
18
19  int main(int argc, char *argv[])
20  {
21      setlocale(LC_ALL, "");
22      // 2.初始化 ROS 节点
23      ros::init(argc, argv, "control");
24      ros::NodeHandle nh;
25      // 3.创建发布者对象
26      ros::Publisher pub = nh.advertise<geometry_msgs::Twist>
27      ("/turtle1/cmd_vel", 1000);
28      // 4.循环发布运动控制消息
29      //4-1.组织消息
30      geometry_msgs::Twist msg;
31      msg.linear.x = 1.0;
32      msg.linear.y = 0.0;
33      msg.linear.z = 0.0;
34
35      msg.angular.x = 0.0;
36      msg.angular.y = 0.0;
37      msg.angular.z = 2.0;
38
39      //4-2.设置发送频率
40      ros::Rate r(10);
41      //4-3.循环发送
42      while (ros::ok())
43      {
44          pub.publish(msg);
45
46          ros::spinOnce();
47      }
48
49      return 0;
```

配置文件此处略

实现方案B: Python

```

1  #! /usr/bin/env python
2  """
3      编写 ROS 节点, 控制小乌龟画圆
4
5      准备工作:
6          1.获取topic(已知: /turtle1/cmd_vel)
7          2.获取消息类型(已知: geometry_msgs/Twist)
8          3.运行前, 注意先启动 turtlesim_node 节点
9
10     实现流程:
11         1.导包
12         2.初始化 ROS 节点
13         3.创建发布者对象
14         4.循环发布运动控制消息
15
16     """
17
18     import rospy
19     from geometry_msgs.msg import Twist
20
21     if __name__ == "__main__":
22         # 2.初始化 ROS 节点
23         rospy.init_node("control_circle_p")
24         # 3.创建发布者对象
25         pub = rospy.Publisher("/turtle1/cmd_vel", Twist, queue_size=1000)
26         # 4.循环发布运动控制消息
27         rate = rospy.Rate(10)
28         msg = Twist()
29         msg.linear.x = 1.0
30         msg.linear.y = 0.0
31         msg.linear.z = 0.0
32         msg.angular.x = 0.0
33         msg.angular.y = 0.0
34         msg.angular.z = 0.5
35
36         while not rospy.is_shutdown():
37             pub.publish(msg)
38             rate.sleep()

```

权限设置以及配置文件此处略

3.运行

首先, 启动 roscore;

然后启动乌龟显示节点;

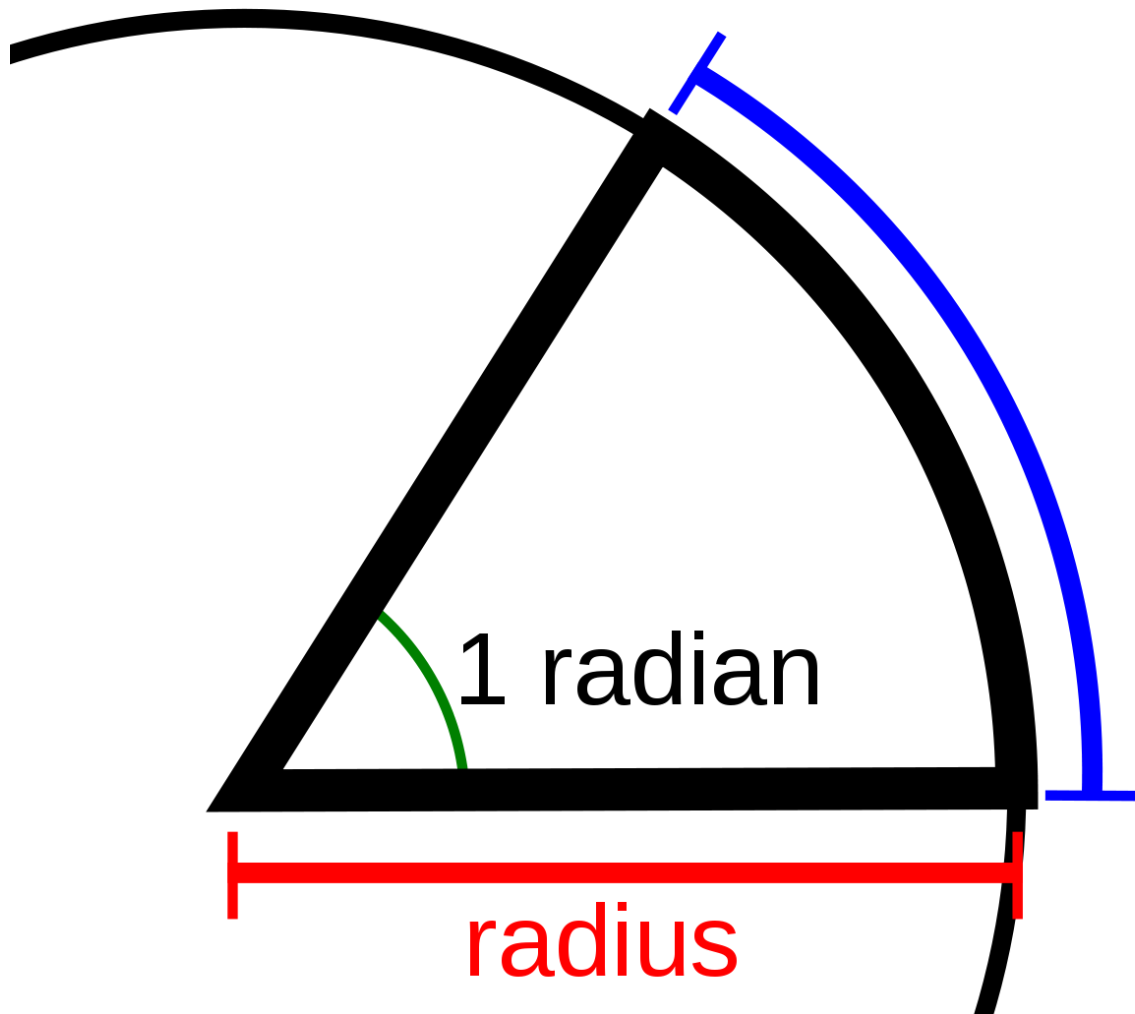
最后执行运动控制节点：

最终执行结果与演示结果类似。

补充资料1:

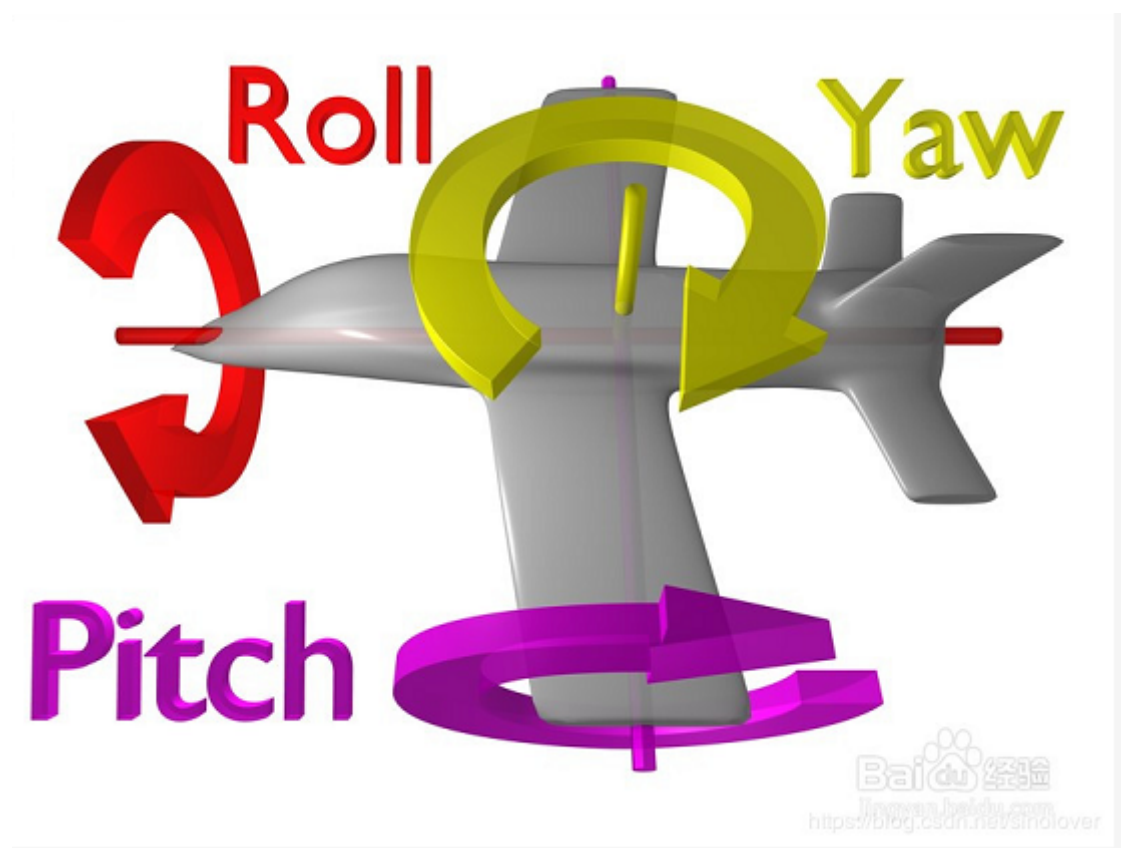
弧度: 单位弧度定义为圆弧长度等于半径时的圆心角。

arc length = radius

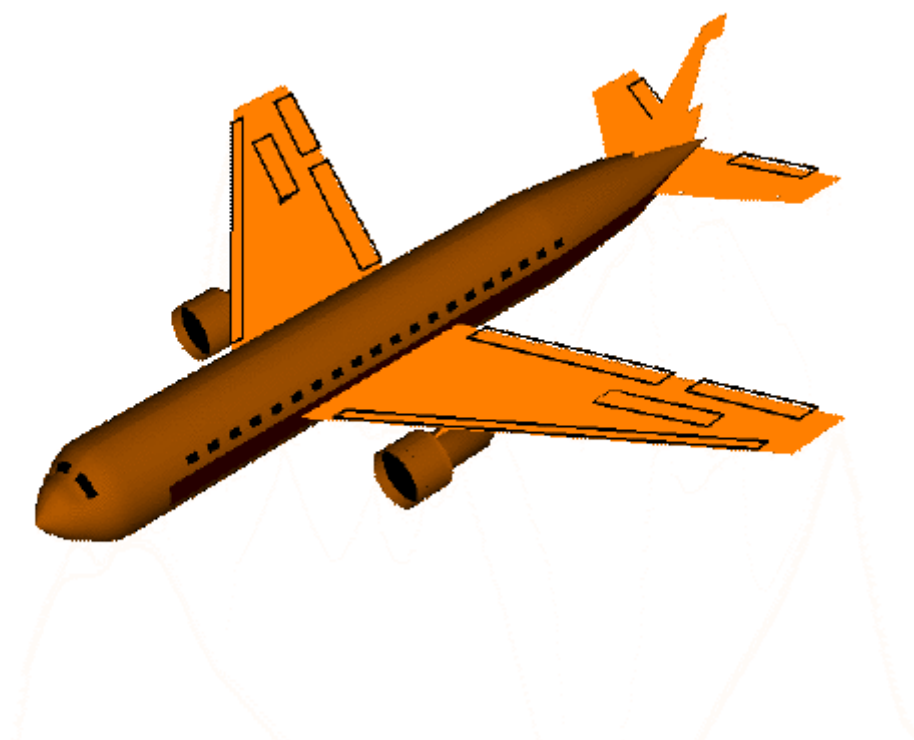


补充资料2:偏航、翻滚与俯仰

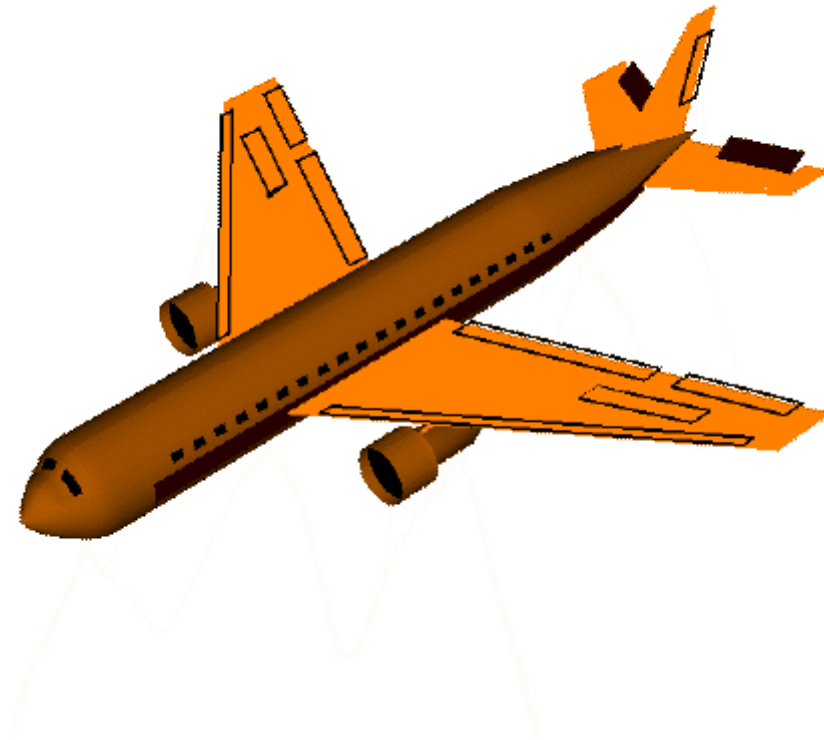
坐标系图解:



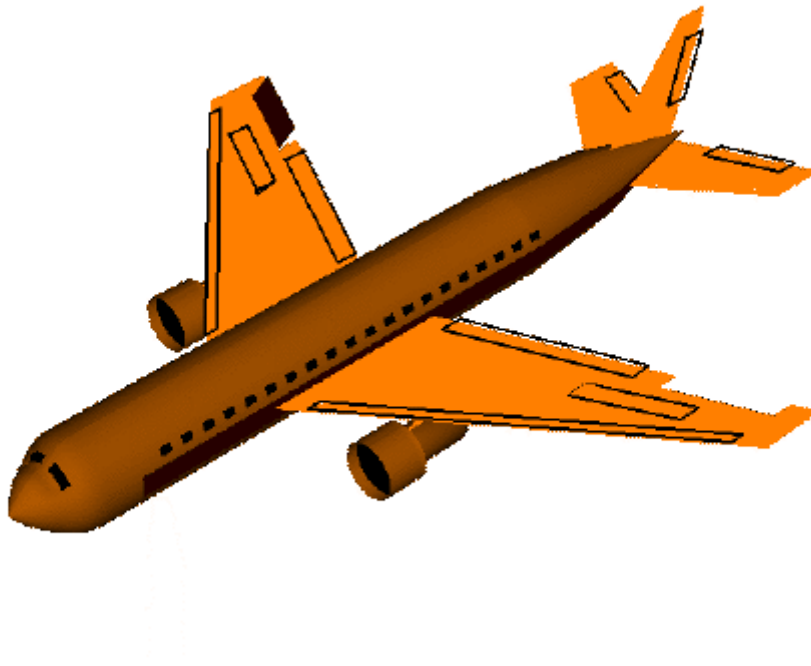
偏航:



俯仰:



翻滚:



2.5.2 实操02_话题订阅

需求描述: 已知turtlesim中的乌龟显示节点，会发布当前乌龟的位姿(窗体中乌龟的坐标以及朝向)，要求控制乌龟运动，并时时打印当前乌龟的位姿。

结果演示:

实现方案A: C++

```
1  /*
2     订阅小乌龟的位姿：时时获取小乌龟在窗体中的坐标并打印
3     准备工作：
4         1.获取话题名称 /turtle1/pose
5         2.获取消息类型 turtlesim/Pose
6         3.运行前启动 turtlesim_node 与 turtle_teleop_key 节点
7
8     实现流程：
9         1.包含头文件
10        2.初始化 ROS 节点
11        3.创建 ROS 句柄
12        4.创建订阅者对象
13        5.回调函数处理订阅的数据
14        6.spin
15 */
16
17 #include "ros/ros.h"
18 #include "turtlesim/Pose.h"
19
20 void doPose(const turtlesim::Pose::ConstPtr& p){
21     ROS_INFO("乌龟位姿信息:x=%.2f,y=%.2f,theta=%.2f,lv=%.2f,av=%.2f",
22             p->x,p->y,p->theta,p->linear_velocity,p->angular_velocity
23     );
24 }
25
26 int main(int argc, char *argv[])
27 {
28     setlocale(LC_ALL,"");
29     // 2.初始化 ROS 节点
30     ros::init(argc,argv,"sub_pose");
31     // 3.创建 ROS 句柄
32     ros::NodeHandle nh;
33     // 4.创建订阅者对象
34     ros::Subscriber sub = nh.subscribe<turtlesim::Pose>
35     ("/turtle1/pose",1000,doPose);
36     // 5.回调函数处理订阅的数据
37     // 6.spin
38     ros::spin();
39     return 0;
40 }
```

配置文件此处略

实现方案B: Python

```
1  #!/usr/bin/env python
2  """
3     订阅小乌龟的位姿：时时获取小乌龟在窗体中的坐标并打印
4     准备工作：
5         1.获取话题名称 /turtle1/pose
```

```

6         2.获取消息类型 turtlesim/Pose
7         3.运行前启动 turtlesim_node 与 turtle_teleop_key 节点
8
9     实现流程:
10        1.导包
11        2.初始化 ROS 节点
12        3.创建订阅者对象
13        4.回调函数处理订阅的数据
14        5.spin
15
16    """
17
18    import rospy
19    from turtlesim.msg import Pose
20
21    def doPose(data):
22        rospy.loginfo("乌龟坐标:x=%.2f,
23        y=%.2f,theta=%.2f",data.x,data.y,data.theta)
24
25    if __name__ == "__main__":
26
27        # 2.初始化 ROS 节点
28        rospy.init_node("sub_pose_p")
29
30        # 3.创建订阅者对象
31        sub = rospy.Subscriber("/turtle1/pose",Pose,doPose,queue_size=1000)
32
33        # 4.回调函数处理订阅的数据
34
35        # 5.spin
36        rospy.spin()

```

权限设置以及配置文件此处略

3.运行

首先，启动 roscore；

然后启动乌龟显示节点，执行运动控制节点；

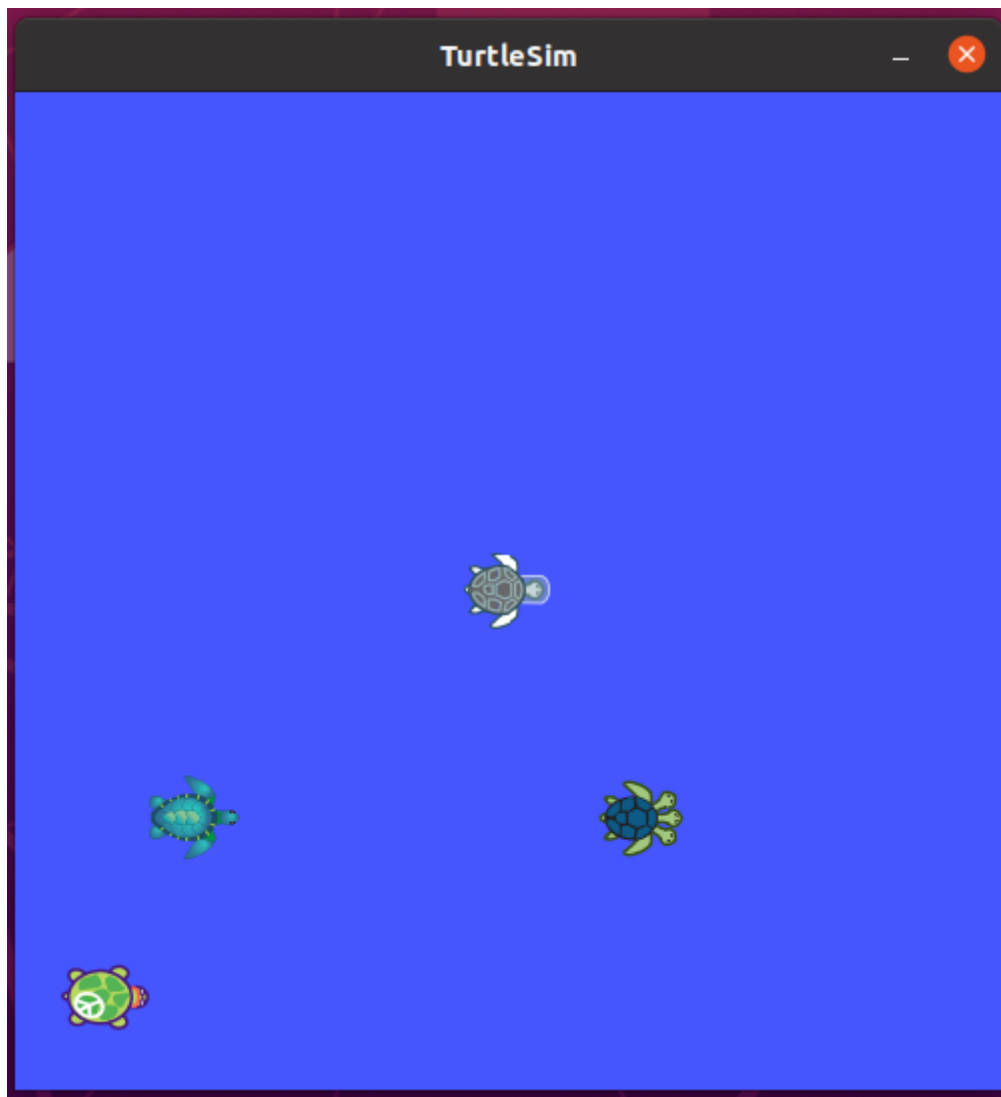
最后启动乌龟位姿订阅节点；

最终执行结果与演示结果类似。

2.5.3 实操03_服务调用

需求描述:编码实现向 turtlesim 发送请求，在乌龟显示节点的窗体指定位置生成一乌龟，这是一个服务请求操作。

结果演示:



实现分析:

1. 首先，需要启动乌龟显示节点。
2. 要通过ROS命令，来获取乌龟生成服务的服务名称以及服务消息类型。
3. 编写服务请求节点，生成新的乌龟。

实现流程:

1. 通过ros命令获取服务与服务消息信息。
2. 编码实现服务请求节点。
3. 启动 roscore、turtlesim_node 、乌龟生成节点，生成新的乌龟。

1.服务名称与服务消息获取

获取话题:/spawn

```
1 | rosservice list
```

获取消息类型:turtlesim/Spawn

```
1 | rosservice type /spawn
```

获取消息格式:

```
1 | rossrv info turtlesim/Spawn
```

响应结果:

```
1 | float32 x
2 | float32 y
3 | float32 theta
4 | string name
5 | ---
6 | string name
```

2.服务客户端实现

创建功能包需要依赖的功能包: roscpp rospy std_msgs turtlesim

实现方案A:C++

```
1 | /*
2 |     生成一只小乌龟
3 |     准备工作:
4 |         1.服务话题 /spawn
5 |         2.服务消息类型 turtlesim/Spawn
6 |         3.运行前先启动 turtlesim_node 节点
7 |
8 |     实现流程:
9 |         1.包含头文件
10 |            需要包含 turtlesim 包下资源, 注意在 package.xml 配置
11 |         2.初始化 ros 节点
12 |         3.创建 ros 句柄
13 |         4.创建 service 客户端
14 |         5.等待服务启动
15 |         6.发送请求
16 |         7.处理响应
17 |
18 | */
19 |
20 | #include "ros/ros.h"
21 | #include "turtlesim/Spawn.h"
22 |
23 | int main(int argc, char *argv[])
24 | {
25 |     setlocale(LC_ALL, "");
26 |     // 2.初始化 ros 节点
27 |     ros::init(argc, argv, "set_turtle");
28 |     // 3.创建 ros 句柄
29 |     ros::NodeHandle nh;
30 |     // 4.创建 service 客户端
31 |     ros::ServiceClient client = nh.serviceClient<turtlesim::Spawn>("/spawn");
32 |     // 5.等待服务启动
33 |     // client.waitForExistence();
```

```

34     ros::service::waitForService("/spawn");
35     // 6.发送请求
36     turtlesim::Spawn spawn;
37     spawn.request.x = 1.0;
38     spawn.request.y = 1.0;
39     spawn.request.theta = 1.57;
40     spawn.request.name = "my_turtle";
41     bool flag = client.call(spawn);
42     // 7.处理响应结果
43     if (flag)
44     {
45         ROS_INFO("新的乌龟生成,名字:%s",spawn.response.name.c_str());
46     } else {
47         ROS_INFO("乌龟生成失败!!!");
48     }
49
50
51     return 0;
52 }

```

配置文件此处略

实现方案B:Python

```

1  #!/usr/bin/env python
2  """
3      生成一只小乌龟
4      准备工作:
5          1.服务话题 /spawn
6          2.服务消息类型 turtlesim/Spawn
7          3.运行前先启动 turtlesim_node 节点
8
9      实现流程:
10         1.导包
11            需要包含 turtlesim 包下资源, 注意在 package.xml 配置
12         2.初始化 ros 节点
13         3.创建 service 客户端
14         4.等待服务启动
15         5.发送请求
16         6.处理响应
17
18     """
19
20     import rospy
21     from turtlesim.srv import Spawn,SpawnRequest,SpawnResponse
22
23     if __name__ == "__main__":
24         # 2.初始化 ros 节点
25         rospy.init_node("set_turtle_p")
26         # 3.创建 service 客户端
27         client = rospy.ServiceProxy("/spawn",Spawn)
28         # 4.等待服务启动

```

```
29     client.wait_for_service()
30     # 5.发送请求
31     req = SpawnRequest()
32     req.x = 2.0
33     req.y = 2.0
34     req.theta = -1.57
35     req.name = "my_turtle_p"
36     try:
37         response = client.call(req)
38         # 6.处理响应
39         rospy.loginfo("乌龟创建成功!, 叫:%s", response.name)
40     except expression as identifier:
41         rospy.loginfo("服务调用失败")
```

权限设置以及配置文件此处略

3.运行

首先，启动 roscore；

然后启动乌龟显示节点；

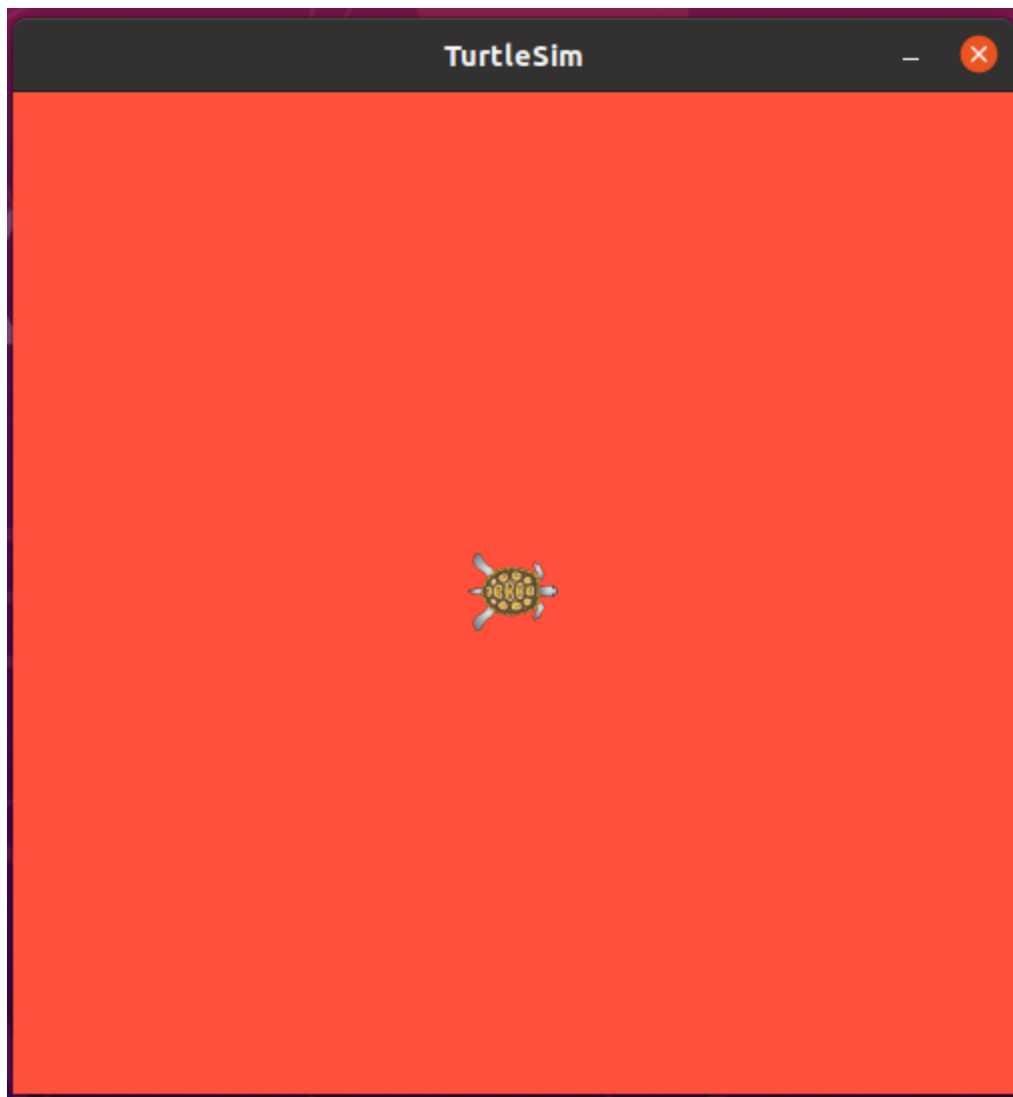
最后启动乌龟生成请求节点；

最终执行结果与演示结果类似。

2.5.4 实操04_参数设置

需求描述: 修改turtlesim乌龟显示节点窗体的背景色，已知背景色是通过参数服务器的方式以 rgb 方式设置的。

结果演示:



实现分析:

1. 首先，需要启动乌龟显示节点。
2. 要通过ROS命令，来获取参数服务器中设置背景色的参数。
3. 编写参数设置节点，修改参数服务器中的参数值。

实现流程:

1. 通过ros命令获取参数。
2. 编码实现参数设置节点。
3. 启动 roscore、turtlesim_node 与参数设置节点，查看运行结果。

1.参数名获取

获取参数列表:

```
1 | rosparam list
```

响应结果:

```
1 | /turtlesim/background_b
2 | /turtlesim/background_g
3 | /turtlesim/background_r
```

2.参数修改

实现方案A:C++

```
1  /*
2     注意命名空间的使用。
3
4  */
5  #include "ros/ros.h"
6
7
8  int main(int argc, char *argv[])
9  {
10     ros::init(argc,argv,"haha");
11
12     ros::NodeHandle nh("turtlesim");
13     //ros::NodeHandle nh;
14
15     // ros::param::set("/turtlesim/background_r",0);
16     // ros::param::set("/turtlesim/background_g",0);
17     // ros::param::set("/turtlesim/background_b",0);
18
19     nh.setParam("background_r",0);
20     nh.setParam("background_g",0);
21     nh.setParam("background_b",0);
22
23
24     return 0;
25 }
```

配置文件此处略

实现方案B:Python

```
1  #! /usr/bin/env python
2
3  import rospy
4
5  if __name__ == "__main__":
6     rospy.init_node("hehe")
7     # rospy.set_param("/turtlesim/background_r",255)
8     # rospy.set_param("/turtlesim/background_g",255)
9     # rospy.set_param("/turtlesim/background_b",255)
10    rospy.set_param("background_r",255)
11    rospy.set_param("background_g",255)
12    rospy.set_param("background_b",255) # 调用时, 需要传入 __ns:=xxx
```

权限设置以及配置文件此处略

3.运行

首先, 启动 roscore;

然后启动背景色设置节点；

最后启动乌龟显示节点；

最终执行结果与演示结果类似。

PS: 注意节点启动顺序，如果先启动乌龟显示节点，后启动背景色设置节点，那么颜色设置不会生效。

4.其他设置方式

方式1:修改小乌龟节点的背景色(命令行实现)

```
1 | rosparam set /turtlesim/background_b 自定义数值
2 | rosparam set /turtlesim/background_g 自定义数值
3 | rosparam set /turtlesim/background_r 自定义数值
```

修改相关参数后，重启 turtlesim_node 节点，背景色就会发生改变了

方式2:启动节点时，直接设置参数

```
1 | rosrn turtlesim turtlesim_node _background_r:=100 _background_g=0
   _background_b=0
```

方式3:通过launch文件传参

```
1 | <launch>
2 |   <node pkg="turtlesim" type="turtlesim_node" name="set_bg"
   output="screen">
3 |     <!-- launch 传参策略1 -->
4 |     <!-- <param name="background_b" value="0" type="int" />
5 |     <param name="background_g" value="0" type="int" />
6 |     <param name="background_r" value="0" type="int" /> -->
7 |
8 |     <!-- launch 传参策略2 -->
9 |     <rosparam command="load" file="$(find
demo03_test_parameter)/cfg/color.yaml" />
10 |   </node>
11 |
12 | </>
```

2.6 通信机制比较

三种通信机制中，参数服务器是一种数据共享机制，可以在不同的节点之间共享数据，话题通信与服务通信是在不同的节点之间传递数据的，三者是ROS中最基础也是应用最为广泛的通信机制。

这其中，话题通信和服务通信有一定的相似性也有本质上的差异，在此将二者做一下简单比较：

二者的实现流程是比较相似的，都是涉及到四个要素：

- 要素1: 消息的发布方/客户端(Publisher/Client)

- 要素2: 消息的订阅方/服务端(Subscriber/Server)
- 要素3: 话题名称(Topic/Service)
- 要素4: 数据载体(msg/srv)

可以概括为: 两个节点通过话题关联到一起, 并使用某种类型的数据载体实现数据传输。

二者的实现也是有本质差异的, 具体比较如下:

	Topic(话题)	Service(服务)
通信模式	发布/订阅	请求/响应
同步性	异步	同步
底层协议	ROSTCP/ROSUDP	ROSTCP/ROSUDP
缓冲区	有	无
时时性	弱	强
节点关系	多对多	一对多(一个 Server)
通信数据	msg	srv
使用场景	连续高频的数据发布与接收:雷达、里程计	偶尔调用或执行某一项特定功能: 拍照、语音识别

不同通信机制有一定的互补性, 都有各自适应的应用场景。尤其是话题与服务通信, 需要结合具体的应用场景与二者的差异, 选择合适的通信机制。

2.7 本章小结

本章主要介绍了ROS中最基本的也是最核心的通信机制实现: 话题通信、服务通信、参数服务器。每种通信机制, 都介绍了如下内容:

- 伊始介绍了当前通信机制的应用场景;
- 介绍了当前通信机制的理论模型;
- 分别介绍了当前通信机制的C++与Python实现。

除此之外, 还介绍了:

- ROS中的常用命令方便操作、调试节点以及通信信息;
- 通过实操又将上述知识点加以整合;
- 最后又着重比较了话题通信与服务通信的相同点以及差异。

掌握本章内容后, 基本上就可以从容应对ROS中大部分应用场景了。

